

Program Analysis - Complete Study Guide

Program Analysis — Complete Study Guide

Lecture 01 — Introduction

Theory

Lecture 02 — Denotational Semantics

Theory

Lecture 03 — Hoare Logic (HL)

Inference Rules

Validity

Important Properties

Lecture 04 — Total Correctness

Partial vs Total Correctness

Total Correctness While Rule

Example: Euclidean Division

Lecture 05 — Incorrectness Logic (IL)

O'Hearn's Triples

IL Inference Rules

Lecture 06 — Real Incorrectness Logic

Key Insight for IL

Lecture 07 — More IL: Non-termination and Control

Non-termination Analysis

Control Flow in IL

Lecture 08 — Symbolic Incorrectness Logic (SIL)

Key Idea

Lecture 09 — Separation Logic (SL)

Key Idea

Lecture 10 — Incorrectness Separation Logic (ISL) and Symbolic Execution Separation Logic (SepSIL)

ISL

SepSIL

Lecture 11 — Abstract Interpretation: Basic Ideas

Core Concept

Lecture 12 — Abstract Interpretation: Formal Foundations

Lattice Theory

Galois Connections

Galois Insertion

Abstract Semantics Soundness

Lecture 13 — Galois Connections

Properties of Galois Connections

Lecture 14 — Abstract Domains

Sign Domain

Lecture 15 — Abstract Analysis (Fixpoint Computation)

Lecture 16 — Local Completeness Logic (LCL)

Completeness in Abstract Interpretation

Lecture 17 — Control Flow Analysis (CFA)

Introduction

Lectures 18–22 — CFA (continued)

CFA for Functional Languages

Taint Analysis via CFA

Lecture 22 — CFA for π -Calculus (0-CFA for Process Algebras)

Motivation

π -Calculus Syntax

Name Mobility Example

Abstract Domains for CFA

Validation Clauses for Actions

Validation Clauses for Processes

Summary of All Validation Clauses

Lecture 23 — CFA for π -Calculus: Examples, Theory, and Polyadic Extension

Worked Example 1: Simple Communication

Worked Example 2: Name Mobility

The Match Clause in Practice

Theoretical Properties

Nature of Imprecision in 0-CFA for π -calculus

Polyadic π -Calculus Extension

Complete Formula Handbook

F-01: Collecting Semantics

F-02: Weakest Liberal Precondition

F-03: Weakest Possible Precondition

F-04: Hoare Triple Validity

F-05: Incorrectness Triple Validity

F-06: Galois Connection

F-07: Soundness of Abstract Semantics

F-08: Best Abstract Correct Approximation

F-09: Completeness Equation

Exam Preparation Guide

Recurring Exercise Patterns

Common Proof Strategies

Common Lattice Constructions

Frequently Tested Concepts

Final Cheat Sheet (Expanded)

1. Denotational Semantics

2. Program Logics — The Four Quadrants

3. Hoare Logic Rules (Partial Correctness)

4. Incorrectness Logic Rules

5. Separation Logic

6. Abstract Interpretation

7. Control Flow Analysis (0-CFA for FUN)

8. 0-CFA for π -Calculus

9. Common Exam Tricks

10. Key Notation Summary

Algorithm Extraction

A-01: Kleene Fixpoint Iteration

A-02: Abstract Fixpoint with Widening

A-03: Worklist Algorithm for Abstract Interpretation

A-04: 0-CFA Constraint Generation

A-05: Interval Widening

A-06: Syntax-Directed CFA ($\models_s$)	
A-07: k -CFA (Call String Approach)	
Complete Exercise Collection	
Source	
Easy	
Medium	
Hard	
Enhanced Formula Handbook	
F-01: Collecting Semantics	
F-02: Weakest Liberal Precondition	
F-03: Weakest Possible Precondition	
F-04: Hoare Triple Validity	
F-05: Incorrectness Triple Validity	
F-06: Galois Connection	
F-07: Soundness of Abstract Semantics	
F-08: Best Abstract Correct Approximation (BCA)	
F-09: Completeness Equation	
F-10: Galois Insertion Property	
F-11: LCL Triple Validity	
F-12: Local Completeness Condition	
F-13: LCL Iteration Rule	
F-14: Fixpoint Approximation Theorem	
F-15: Abstract Semantics of Iteration	
F-16: Correctness Condition for $F^\#$	
F-17: Abstract Domain Operations Tables	
F-18: Interval Domain Operations	
F-19: Congruence Domain Operations	
F-20: Interval Widening	
Lecture Summaries	
Lecture 01 — Introduction	
Lecture 02 — Denotational Semantics	
Lecture 03 — Hoare Logic	
Lecture 04 — Total Correctness	
Lecture 05 — Incorrectness Logic	
Lecture 06 — Real Incorrectness Logic	
Lecture 07 — More IL: Non-termination and Control	
Lecture 08 — Symbolic Incorrectness Logic (SIL)	
Lecture 09 — Separation Logic	
Lecture 10 — ISL and SepSIL	
Lecture 11 — Abstract Interpretation: Basic Ideas	
Lecture 12 — Abstract Interpretation: Formal Foundations	
Lecture 13 — Galois Connections (Advanced)	
Lecture 14 — Abstract Numerical Domains	
Lecture 15 — Abstract Analysis	
Lecture 16 — Local Completeness Logic	
Lectures 17–21 — Control Flow Analysis (FUN)	
Lecture 22 — 0-CFA for π -Calculus	
Lecture 23 — CFA for π -Calculus: Theory and Extensions	
Cross-Reference Index	
Formulas by Lecture	
Exercises by Topic	
Algorithms by Topic	
Validation Report	
LaTeX Validation	

Program Analysis — Complete Study Guide

Course: Program Analysis 2025-26

Teachers: Chiara Bodei, Roberto Bruni, Roberta Gori

University: Università di Pisa

Lecture 01 — Introduction

Theory

Software Verification

- **Correctness:** proving the absence of bugs (over-approximation)
- **Incorrectness:** proving the presence of bugs (under-approximation)

Semantics

Memory states:

$$\Sigma \triangleq \{\sigma: X \rightarrow \mathbb{Z}\}$$

Forward semantics (deterministic): $\llbracket c \rrbracket: \Sigma \rightarrow \Sigma_{\perp}$ where $\Sigma_{\perp} = \Sigma \uplus \{\perp\}$

Collecting semantics: $\llbracket c \rrbracket: \wp(\Sigma) \rightarrow \wp(\Sigma)$

$$\llbracket c \rrbracket P = \bigcup_{\sigma \in P} \llbracket c \rrbracket \sigma$$

Undecidability

Rice's Theorem: Let $\mathcal{P}(c)$ be a non-trivial semantic property of programs. There exists no algorithm such that, for every program c , it returns true iff $\mathcal{P}(c)$ holds.

| No analysis method that is **automatic**, **universal**, and **exact**!

Approximation Spectrum

Technique	Automatic	Sound	Complete
Testing	Yes	No	Yes

Technique	Automatic	Sound	Complete
Machine-assisted proving	No	Yes	Quasi
Model checking	Yes	Yes*	Yes*
Conservative static analysis (over-approx)	Yes	Yes	No
Bug finding (under-approx)	Yes	No	No

Soundness and Completeness

- **Soundness:** $\text{analysis}(c) = \text{true} \Rightarrow \mathcal{P}(c)$
- **Completeness:** $\mathcal{P}(c) \Rightarrow \text{analysis}(c) = \text{true}$

Over-approximation vs Under-approximation

- **Over-approximation:** Good for proving correctness (true negatives), bad for bug-finding (false positives)
- **Under-approximation:** Good for bug-finding (true positives), bad for proving correctness (false negatives)

Lecture 02 — Denotational Semantics

Theory

Concrete Domain

$$\Sigma \triangleq \{\sigma: X \rightarrow \mathbb{Z}\} \quad \wp(\Sigma) \triangleq \{P \mid P \subseteq \Sigma\}$$

State Notation

- $\sigma = [x \mapsto 2]$ — variable x has value 2, all others 0
- $\sigma[n/x]$ — state update: x holds n , any other y holds $\sigma(y)$
- $(x = 1, y = 2)$ — property notation: set of all states where $x = 1$ and $y = 2$

Regular Commands Syntax

$$\begin{aligned} c &::= e \mid c_1; c_2 \mid c_1 + c_2 \mid c^* \\ e &::= \text{skip} \mid x := a \mid b? \mid \dots \\ a &::= n \mid x \mid a_1 + a_2 \mid \dots \\ b &::= a_1 \leq a_2 \mid b_1 \wedge b_2 \mid \dots \end{aligned}$$

Collecting Semantics

$$\llbracket c \rrbracket: \wp(\Sigma) \rightarrow \wp(\Sigma)$$

Atomic commands: - $\llbracket \text{skip} \rrbracket P \triangleq P$ - $\llbracket x := a \rrbracket P \triangleq \{\sigma[n/x] \mid \sigma \in P, n = \llbracket a \rrbracket \sigma\}$ - $\llbracket b? \rrbracket P \triangleq \llbracket b \rrbracket P$ (states in P satisfying b)

Compound commands: - $\llbracket c_1; c_2 \rrbracket P \triangleq \llbracket c_2 \rrbracket (\llbracket c_1 \rrbracket P)$ - $\llbracket c_1 + c_2 \rrbracket P \triangleq \llbracket c_1 \rrbracket P \cup \llbracket c_2 \rrbracket P$ - $\llbracket c^* \rrbracket P \triangleq \bigcup_{k=0}^{\infty} \llbracket c \rrbracket^k P$

Encoding conditionals and loops: - if b then c_1 else $c_2 \triangleq (b?; c_1) + (\neg b?; c_2)$ - while b do $c \triangleq (b?; c)^*; \neg b?$

Partial Correctness

$$\llbracket c \rrbracket P \subseteq Q$$

Every reachable state from P is in Q (non-termination allowed).

Dijkstra's Weakest Liberal Precondition (wlp)

$$\text{wlp}(c, Q) = \{\sigma \mid \llbracket c \rrbracket \{\sigma\} \subseteq Q\}$$

Property: $P \subseteq \text{wlp}(c, Q) \Leftrightarrow \llbracket c \rrbracket P \subseteq Q$

Relational Semantics

$$\llbracket c \rrbracket \subseteq \Sigma \times \Sigma \text{ where } \llbracket c \rrbracket = \{(\sigma, \delta) \mid \sigma \in \Sigma, \delta \in \llbracket c \rrbracket \sigma\}$$

Backward Semantics

$$\llbracket \bar{c} \rrbracket = \{(\delta, \sigma) \mid (\sigma, \delta) \in \llbracket c \rrbracket\}$$

Properties: $\delta \in \llbracket c \rrbracket \sigma \Leftrightarrow \sigma \in \llbracket \bar{c} \rrbracket \delta$

Hoare's Weakest Possible Precondition (wpp)

$$\text{wpp}(c, Q) = \{\sigma \mid \llbracket c \rrbracket \sigma \cap Q \neq \emptyset\}$$

$$\llbracket \bar{c} \rrbracket Q = \text{wpp}(c, Q)$$

wpp vs wlp

- $\text{wpp}(c, Q)$: largest set of input states that have a **successful** computation
- $\text{wlp}(c, Q)$: largest set of input states whose computations are **all** successful

Lecture 03 — Hoare Logic (HL)

Inference Rules

Skip Axiom

$$\overline{\{P\} \text{ skip } \{P\}}$$

Assignment Axiom (Hoare)

$$\overline{\{Q[a/x]\} x := a \{Q\}}$$

Assignment Axiom (Floyd)

$$\overline{\{P\} x := a \{ \exists x'. P[x'/x] \wedge x = a[x'/x] \}}$$

Composition Rule

$$\frac{\{P\} c_1 \{R\} \quad \{R\} c_2 \{Q\}}{\{P\} c_1 ; c_2 \{Q\}}$$

Conditional Rule

$$\frac{\{P \wedge b\} c_1 \{Q\} \quad \{P \wedge \neg b\} c_2 \{Q\}}{\{P\} \text{ if } b \text{ then } c_1 \text{ else } c_2 \{Q\}}$$

While Rule

$$\frac{\{P \wedge b\} c \{P\}}{\{P\} \text{ while } b \text{ do } c \{P \wedge \neg b\}}$$

P is the **loop invariant**.

Consequence Rule

$$\frac{P \Rightarrow P' \quad \{P'\} c \{Q'\} \quad Q' \Rightarrow Q}{\{P\} c \{Q\}}$$

Validity

A HL triple $\{P\} c \{Q\}$ is **valid** iff $\llbracket c \rrbracket P \subseteq Q$.

Important Properties

- **Soundness:** Any derivable HL triple is valid (proof by induction on derivation tree)
 - **Finding invariants is difficult** — requires creativity
-

Lecture 04 — Total Correctness

Partial vs Total Correctness

- **Partial:** $\{P\} c \{Q\}$ — precondition met $\Rightarrow$ postcondition holds (if termination occurs)
- **Total:** $[P] c [Q]$ — precondition met $\Rightarrow$ **termination** + postcondition holds
- Total correctness = partial correctness + termination

Total Correctness While Rule

$$\frac{\{P \wedge b\} c \{P\} \quad \{P \wedge b \wedge t = z\} c \{t < z\} \quad P \Rightarrow t \geq 0}{\{P\} \text{ while } b \text{ do } c \{P \wedge \neg b\}}$$

Where: - t is the **variant** (termination function) — an arithmetic expression that decreases at each iteration and is bounded below - z is a fresh variable that stores the initial value of t

Example: Euclidean Division

Total correctness proof with variant $t \triangleq r$: - $P \Rightarrow t \geq 0$:
 $(r \geq 0 \wedge y > 0 \wedge x = r + qy) \Rightarrow r \geq 0 \checkmark$ - Body decreases variant:
 $\{r \geq y > 0 \wedge \dots \wedge r = z\} r := r - y; q := q + 1 \{r < z\} \checkmark$

Lecture 05 — Incorrectness Logic (IL)

O'Hearn's Triples

$$[P] c [Q] \Leftrightarrow \llbracket c \rrbracket P \supseteq Q$$

Meaning: Any output in the postcondition is reachable from some input in the precondition.

First-order Formulas

- HL: $\llbracket c \rrbracket P \subseteq Q \equiv \forall \sigma \in P. \forall \delta \in \llbracket c \rrbracket \sigma. \delta \in Q$
- IL: $\llbracket c \rrbracket P \supseteq Q \equiv \forall \delta \in Q. \exists \sigma \in P. \delta \in \llbracket c \rrbracket \sigma$

IL Inference Rules

Skip

$$\overline{[P] \text{ skip } [P]}$$

Assignment (Floyd) — Same as HL!

$$\overline{[P] x := a \ [\exists x' . P[x' / x] \wedge x = a[x' / x]]}$$

Note: Hoare's axiom $[Q[a/x]] x := a \ [Q]$ is **unsound** for IL!

Assume

$$\overline{[P] b? \ [P \wedge b]}$$

Error

$$\overline{[P] \text{ error() } [\text{false}]}$$

Nondet

$$\overline{[P] x := \text{nondet() } [\exists x. P]}$$

Sequence — Same as HL

$$\frac{[P] c_1 \ [R] \quad [R] c_2 \ [Q]}{[P] c_1 ; c_2 \ [Q]}$$

Choice

$$\frac{[P] c_1 \ [Q_1] \quad [P] c_2 \ [Q_2]}{[P] c_1 + c_2 \ [Q_1 \vee Q_2]}$$

Consequence — Reverse of HL!

$$\frac{P' \Rightarrow P \quad [P'] c \ [Q'] \quad Q \Rightarrow Q'}{[P] c \ [Q]}$$

For IL: Shrink the postcondition (scalable bug detection)

Choice Rules (Dropping Disjuncts)

$$\frac{[P] c_1 \ [Q]}{[P] c_1 + c_2 \ [Q]} \quad \frac{[P] c_2 \ [Q]}{[P] c_1 + c_2 \ [Q]}$$

If-then-else (Derived)

$$\frac{[P \wedge b] c_1 [Q_1] \quad [P \wedge \neg b] c_2 [Q_2]}{[P] \text{ if } b \text{ then } c_1 \text{ else } c_2 [Q_1 \vee Q_2]}$$

Loop: Bounded Unrolling

$$\frac{[P] c^*; c [Q]}{[P] c^* [Q]} \quad \frac{}{[P] c^* [P]}$$

Backwards Variant (Weak)

$$\frac{\forall n \in \mathbb{N}. [P_n] c [P_{n+1}]}{[P_0] c^* [\exists k. P_k]}$$

Principle of Agreement

If $[P'] c [Q']$ and $P' \Rightarrow P$ and $\{P\} c \{Q\}$, then $Q' \Rightarrow Q$.

Principle of Denial

If $[P'] c [Q']$ and $P' \Rightarrow P$ and $\neg(Q' \Rightarrow Q)$, then $\neg(\{P\} c \{Q\})$.

Lecture 06 — Real Incorrectness Logic

See Lecture 05 — IL. This lecture covers the practical application of Incorrectness Logic with realistic programming language features, including:

- **Non-determinism:** $x := \text{nondet}()$
- **Error detection:** `error()` kills execution path
- **Loop unrolling in practice:** finite unrolling c^n for bug detection

Key Insight for IL

HL	IL
You get to forget information as you go along a path, but you must remember all the paths	You must remember information as you go along a path, but you get to forget some of the paths

Lecture 07 — More IL: Non-termination and Control

Non-termination Analysis

- **Over-approximation** of reachable states: find $Q \supseteq \llbracket c \rrbracket \sigma$ such that $Q \subseteq \emptyset \rightarrow$ proves non-termination
- **Under-approximation** of reachable states: find $Q \subseteq \llbracket c \rrbracket \sigma$ such that $Q \supset \emptyset \rightarrow$ proves termination

Control Flow in IL

- **Guarded commands:** $b?$ acts as filter
 - **Non-deterministic choice:** $+$ represents branching
 - **Encoding errors:** `error()` has empty semantics $\llbracket \text{error}() \rrbracket P = \emptyset$
-

Lecture 08 — Symbolic Incorrectness Logic (SIL)

Key Idea

SIL extends IL with **symbolic execution** to handle: - Complex data structures - Path-sensitive analysis - Compositional reasoning

Symbolic States

Instead of concrete states $\sigma: X \rightarrow \mathbb{Z}$, SIL uses **symbolic states** with: - Path conditions (formulas on input variables) - Symbolic variable bindings

SIL Rules

Similar to IL but with symbolic substitutions and path constraints carried along through execution paths.

Lecture 09 — Separation Logic (SL)

Key Idea

SL extends Hoare Logic to reason about **heap-manipulating programs** (pointers, mutable data structures).

Heap Model

- **Stack:** variables mapped to values $\sigma: \text{Var} \rightarrow \text{Val}$
- **Heap:** memory locations mapped to values $h: \text{Loc} \rightarrow_{\text{fin}} \text{Val}$
- **State:** (σ, h)

Separation Connectives

- **Separating conjunction:** $P \star Q$ — P and Q hold for **disjoint** portions of the heap
- **Separating implication (magic wand):** $P \multimap Q$ — if a disjoint heap satisfying P is added, then Q holds

Key SL Rules

$$\frac{\overline{\{Q[e/x]\} \ x := e \ \{Q\}}}{\{x \hookrightarrow -\} \ \text{dispose}(x) \ \{\text{true}\}}$$

Frame Rule:

$$\frac{\{P\} \ c \ \{Q\}}{\{P \star R\} \ c \ \{Q \star R\}}$$

where c does not modify free variables of R .

Lecture 10 — Incorrectness Separation Logic (ISL) and Symbolic Execution Separation Logic (SepSIL)

ISL

ISL combines Incorrectness Logic with Separation Logic for **bug finding in heap-manipulating programs**.

Key idea: Under-approximate reasoning + separation logic for heap.

SepSIL

Symbolic execution with separation logic for bug finding: - Tracks symbolic heap configurations - Under-approximate: only reachable states are reported - Can detect memory errors (dangling pointers, double free, leaks)

Lecture 11 — Abstract Interpretation: Basic Ideas

Core Concept

Abstract interpretation provides a **systematic framework** for designing sound static analyses:

1. Start with **concrete semantics** $\llbracket c \rrbracket: \wp(\Sigma) \rightarrow \wp(\Sigma)$
2. Choose an **abstract domain** modeling properties of interest
3. Define **abstract semantics** $\llbracket c \rrbracket^\#$ that executes commands on the abstract domain
4. Prove **correctness**: the abstract semantics soundly approximates the concrete
5. Compute fixpoints for loops

Why Abstraction?

Due to Rice's Theorem, we cannot have exact automatic analysis. We must: - Trade precision for decidability - Accept "don't know" answers (sound over-approximation)

Abstraction Levels

Concrete: infinite set of states

↓

Abstract: finite/simple property (e.g., sign of variable)

Soundness principle: If the analysis says YES, the property definitely holds.

Lecture 12 — Abstract Interpretation: Formal Foundations

Lattice Theory

A **complete lattice** $(L, \sqsubseteq, \perp, \top, \sqcup, \sqcap)$: - $\sqsubseteq$: partial order - $\perp$: least element - $\top$: greatest element - $\sqcup$: least upper bound (join) - $\sqcap$: greatest lower bound (meet)

Concrete Domain

$(\wp(\Sigma), \subseteq, \emptyset, \Sigma, \cup, \cap)$ is a complete lattice.

Galois Connections

A **Galois connection** $(\mathcal{C}, \alpha, \gamma, A)$ between concrete domain $\mathcal{C}$ and abstract domain A :

$$\alpha(c) \sqsubseteq_A a \Leftrightarrow c \sqsubseteq_{\mathcal{C}} \gamma(a)$$

- $\alpha: \mathcal{C} \rightarrow A$ — abstraction function
- $\gamma: A \rightarrow \mathcal{C}$ — concretization function

Properties: - α and γ are monotone - $\gamma \circ \alpha$ is extensive ($x \sqsubseteq \gamma(\alpha(x))$) - $\alpha \circ \gamma$ is reductive ($\alpha(\gamma(y)) \sqsubseteq y$)

Galois Insertion

A Galois insertion adds: $\alpha \circ \gamma = \text{id}_A$ (no redundant abstract elements).

Abstract Semantics Soundness

For each atomic command:

$$\alpha \circ \llbracket c \rrbracket \sqsubseteq \llbracket c \rrbracket^\# \circ \alpha$$

Or equivalently:

$$\llbracket c \rrbracket \circ \gamma \sqsubseteq \gamma \circ \llbracket c \rrbracket^\#$$

Fixpoint Approximation

For loops: $\text{lfp}(F) \sqsubseteq \gamma(\text{lfp}(F^\#))$ where F and $F^\#$ are the concrete and abstract transformers.

Kleene fixpoint theorem: $\text{lfp}(F) = \bigsqcup_{n \geq 0} F^n(\perp)$

Widening

To ensure termination of fixpoint computation: - **Widening operator** ∇ : accelerates convergence - $x \nabla y \sqsupseteq x \sqcup y$ - Every ascending chain stabilizes after finitely many widenings

Lecture 13 — Galois Connections

Properties of Galois Connections

1. α preserves arbitrary joins: $\alpha(\bigsqcup X) = \bigsqcup \alpha(X)$
2. γ preserves arbitrary meets: $\gamma(\bigsqcap Y) = \bigsqcap \gamma(Y)$
3. α is uniquely determined by γ and vice versa

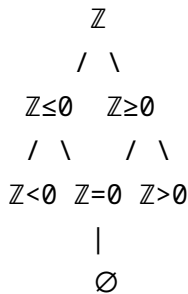
Closure Operators

$A \circ \gamma$ is a **closure operator**: - Monotone: $x \sqsubseteq y \Rightarrow \rho(x) \sqsubseteq \rho(y)$ - Extensive: $x \sqsubseteq \rho(x)$ - Idempotent: $\rho(\rho(x)) = \rho(x)$

In a Galois insertion, $\rho = \gamma \circ \alpha$ is the **closure**.

Lecture 14 — Abstract Domains

Sign Domain



Where: - $\perp = \emptyset$ (impossible) - $\top = \mathbb{Z}$ (any value) - $\mathbb{Z}_{\geq 0} = \{n \mid n \geq 0\}$ - $\mathbb{Z}_{\leq 0} = \{n \mid n \leq 0\}$ - $\mathbb{Z}_{> 0} = \{n \mid n > 0\}$ - $\mathbb{Z}_{< 0} = \{n \mid n < 0\}$ - $\mathbb{Z}_{= 0} = \{0\}$

Relational vs Non-relational Domains

- **Non-relational**: Track each variable independently (e.g., Sign, Interval)
- **Relational**: Track relationships between variables (e.g., Polyhedra, Octagons)

Domain	Expressiveness	Cost
Sign	Signs of variables	$O(n)$
Interval	$x \in [l, u]$	$O(n)$
Octagon	$\pm x \pm y \leq c$	$O(n^2)$
Polyhedron	$\sum a_i x_i \leq c$	$O(2^n)$
Congruence	$x \equiv a \pmod{m}$	$O(n)$

Lecture 15 — Abstract Analysis (Fixpoint Computation)

Computing Abstract Semantics

1. Build **control flow graph** (CFG) with program points
2. Associate **abstract invariant** with each program point
3. Compute **least fixpoint** of the abstract semantic function

Worklist Algorithm

Initialize all program points to $\perp$ (or initial values for entry point).

Iterate: 1. Pick a program point whose abstract value changed 2. Apply abstract transformer to successors 3. Join new values with existing abstract values 4. If any successor's value changed, add to worklist

Until worklist is empty (fixpoint reached).

Widening in Practice

At loop head: merge old and new abstract values using ∇ to ensure termination.

Example (Interval widening):

$$[l_1, u_1] \nabla [l_2, u_2] = [\text{if } l_2 < l_1 \text{ then } -\infty \text{ else } l_1, \text{ if } u_2 > u_1 \text{ then } +\infty \text{ else } u_1]$$

Lecture 16 — Local Completeness Logic (LCL)

Completeness in Abstract Interpretation

Best abstract correct approximation (bca):

$$\llbracket c \rrbracket_A^\# = \alpha \circ \llbracket c \rrbracket \circ \gamma$$

Completeness equation:

$$\forall P. \alpha(\llbracket c \rrbracket P) = \alpha(\llbracket c \rrbracket (\gamma(\alpha(P))))$$

Or equivalently:

$$\alpha \circ \llbracket c \rrbracket = \alpha \circ \llbracket c \rrbracket \circ \gamma \circ \alpha$$

Local Completeness

A triple $\{P\} c \{Q\}$ is **locally complete** if the completeness equation holds when starting from P .

LCL Triples

$$\langle P \rangle c \langle Q \rangle$$

where α_P is the abstraction relative to property P .

Lecture 17 — Control Flow Analysis (CFA)

Introduction

CFA is a **constraint-based** static analysis technique for higher-order programs (functional languages, π -calculus).

Goal

Given a program, determine **which functions may be called at each call site** and **which values may flow to which variables**.

0-CFA

The simplest CFA — **monovariant** (one abstract value per program point regardless of calling context).

Algorithm: 1. Assign each subexpression a label 2. Generate constraints based on program structure 3. Solve constraints via fixpoint computation

Constraint Generation

For a functional language with: - Variables: ℓ_x — set of possible values for x - Abstractions (functions): $\text{fun } f(x) = e$ at label ℓ generates $\ell \in \ell_f$ - Applications: $e_1 \ e_2$ at label ℓ generates constraints linking function results to application point

Lectures 18–22 — CFA (continued)

CFA for Functional Languages

Constraint Rules

For expression e with label ℓ :

- $[[x]]^\ell : \ell_x \supseteq \{\ell\}$ (variable reference)
- $[[\lambda x.e]]^\ell : \{\text{closure}(\ell', \rho)\} \in \ell$ where ℓ' labels $\lambda x.e$
- $[[e_1 e_2]]^\ell : \text{if } \text{closure}(\ell', \rho) \in \ell_{e_1} \text{ then } \ell_{e_2} \supseteq \rho(x) \dots$

Flow Logic Approach

CFA can be formulated as a **flow logic**: 1. Write down **acceptability constraints** that a valid analysis must satisfy 2. Show that the constraints have a **least solution** (the most precise analysis) 3. Compute the solution via fixpoint iteration

Abstract Domains for CFA

- **Cache**: maps labels to sets of abstract values
- **Abstract values**: closures, constructors, atomic values
- **Abstract environment**: maps variables to abstract values

Taint Analysis via CFA

CFA can be extended for **taint analysis**: - Track which values are “tainted” (from user input) - Propagate taint through program - Detect if tainted data reaches sensitive operations (sinks)

Lecture 22 — CFA for π -Calculus (0-CFA for Process Algebras)

Motivation

- In CCS, communication is statically defined: channels cannot be transmitted or received
- The π -calculus extends CCS by allowing channels to be treated as ordinary values
- This enables **name mobility**: channels themselves can be communicated, making the communication structure dynamic
- CFA must predict which values may flow over which channels, even when channel names are passed at runtime

π -Calculus Syntax

Actions (Prefixes)

$$\begin{aligned}
 \pi &::= \bar{u}\langle v \rangle && (\text{send } v \text{ on } u) \\
 &| u(x) && (\text{receive on } u, \text{ store in } x) \\
 &| \tau && (\text{internal action})
 \end{aligned}$$

Processes

$P ::= 0$	(inactive process)
$ \pi.P$	(action prefix)
$ [x = y]P$	(matching: if $x = y$ then P)
$ (\nu n)P$	(restriction: n is fresh/local)
$ \sum_{i \in I} \pi_i . P_i$	(non-deterministic guarded choice)
$ P_1 \mid P_2$	(parallel composition)
$!P$	(replication: $P \mid P \mid P \mid \dots$)

Reduction Semantics (Key Rule)

$$[\text{Com}] \quad (\bar{n}(m).P + P') \mid (n(x).Q + Q') \rightarrow P \mid Q[m/x]$$

A process can also reduce under restriction, parallel composition, and structural congruence.

Name Mobility Example

$A = (\nu ab)(\bar{ab}(m).A')$ -- A sends message m on channel ab to B
 $B = ab(x).B'$ -- B receives on ab
 S -- secure server that establishes channels

System: $(\nu as)(\nu bs)((\nu ab) A \mid B \mid S)$

Channel ab is created by A, used for communication, and can be passed to other processes.

Abstract Domains for CFA

Domain Definitions

$$\begin{aligned} \text{Val} &= \wp(\text{Const}) \\ \rho &: \text{Var} \rightarrow \wp(\text{Const}) \\ \kappa &: \text{Var} \rightarrow \wp(\text{Const}) \end{aligned}$$

- ρ (**abstract environment**): maps each variable to the set of constants it may be bound to
- κ (**abstract channel environment**): maps each channel constant to the set of constants that may be communicated over it
- Both domains form complete lattices under pointwise subset ordering
- Extension to constants: $\rho(n) = \{n\}$ for any constant n

CFA Guess

A **guess** (ρ, κ) is a proposed analysis result. It is **acceptable** if it safely over-approximates all runtime behaviour.

Validation Clauses for Actions

Output: $\bar{u}\langle v \rangle$

$$[\text{out}] \quad (\rho, \kappa) \models_A \bar{u}\langle v \rangle \Leftrightarrow \forall n \in \rho(u): \rho(v) \subseteq \kappa(n)$$

The clause generalizes three subcases: - $\bar{n}\langle m \rangle$ (both constants): $m \subseteq \kappa(n)$ - $\bar{n}\langle v \rangle$ (constant channel): $\rho(v) \subseteq \kappa(n)$ - $\bar{u}\langle m \rangle$ (constant value): $\forall n \in \rho(u): m \subseteq \kappa(n)$

Intuition: whatever v may evaluate to must be recorded as possibly communicated on every channel that u may denote.

Input: $u(x)$

$$[\text{in}] \quad (\rho, \kappa) \models_A u(x) \Leftrightarrow \forall n \in \rho(u): \kappa(n) \subseteq \rho(x)$$

Subcase $n(x)$: $\kappa(n) \subseteq \rho(x)$.

Intuition: anything that may be communicated over any channel u denotes may be received and bound to x .

Internal Action: τ

$$[\text{tau}] \quad (\rho, \kappa) \models_A \tau$$

Always satisfied (no communication effect to track).

Validation Clauses for Processes

Inactive Process

$$[\text{nil}] \quad (\rho, \kappa) \models_P 0$$

Always satisfied.

Prefix

$$[\text{pref}] \quad (\rho, \kappa) \models_P \pi.P \Leftrightarrow (\rho, \kappa) \models_A \pi \wedge (\rho, \kappa) \models_P P$$

Parallel Composition

$$[\text{par}] \quad (\rho, \kappa) \models_P P_1 \mid P_2 \Leftrightarrow (\rho, \kappa) \models_P P_1 \wedge (\rho, \kappa) \models_P P_2$$

Sum (Non-deterministic Choice)

$$[\text{sum}] \quad (\rho, \kappa) \models_P \sum_{i \in I} \pi_i . P_i \Leftrightarrow \forall i \in I: ((\rho, \kappa) \models_A \pi_i \wedge (\rho, \kappa) \models_P P_i)$$

Matching

$$[\text{match}] \quad (\rho, \kappa) \models_P [x = y]P \Leftrightarrow (\rho, \kappa) \models_P P$$

The analysis is **oblivious** to matching by default: it over-approximates by assuming the match could always succeed.

Refined match clause (more precise):

$$[\text{match}^*] \quad (\rho, \kappa) \models_P [x = y]P \Leftrightarrow \rho(x) \cap \rho(y) \neq \emptyset \Rightarrow (\rho, \kappa) \models_P P$$

If the two variables' abstract values have no common elements, the branch is unreachable and can be skipped.

Restriction

$$[\text{res}] \quad (\rho, \kappa) \models_P (\nu n)P \Leftrightarrow (\rho, \kappa) \models_P P$$

The analysis is oblivious to scoping: names are treated as global constants.

Replication

$$[\text{rep}] \quad (\rho, \kappa) \models_P !P \Leftrightarrow (\rho, \kappa) \models_P P$$

Replication is treated as an unbounded number of copies (all analysed with the same (ρ, κ)).

Summary of All Validation Clauses

Construct	Clause
[nil]	$(\rho, \kappa) \models_P 0$ always
[pref]	$(\rho, \kappa) \models_P \pi . P \Leftrightarrow (\rho, \kappa) \models_A \pi \wedge (\rho, \kappa) \models_P P$
[par]	$(\rho, \kappa) \models_P P_1 \mid P_2 \Leftrightarrow (\rho, \kappa) \models_P P_1 \wedge (\rho, \kappa) \models_P P_2$
[sum]	$(\rho, \kappa) \models_P \sum_i \pi_i . P_i \Leftrightarrow \forall i: (\rho, \kappa) \models_A \pi_i \wedge (\rho, \kappa) \models_P P_i$
[match]	$(\rho, \kappa) \models_P [x = y]P \Leftrightarrow (\rho, \kappa) \models_P P$
[res]	$(\rho, \kappa) \models_P (\nu n)P \Leftrightarrow (\rho, \kappa) \models_P P$
[rep]	$(\rho, \kappa) \models_P !P \Leftrightarrow (\rho, \kappa) \models_P P$
[out]	$(\rho, \kappa) \models_A \bar{u}\langle v \rangle \Leftrightarrow \forall n \in \rho(u): \rho(v) \subseteq \kappa(n)$
[in]	$(\rho, \kappa) \models_A u(x) \Leftrightarrow \forall n \in \rho(u): \kappa(n) \subseteq \rho(x)$
[tau]	$(\rho, \kappa) \models_A \tau$ always

Lecture 23 — CFA for π -Calculus: Examples, Theory, and Polyadic Extension

Worked Example 1: Simple Communication

Process:

$$(\bar{a}(b).P' + \bar{a}(d).P'') \mid a(w).\bar{c}(w).Q' \mid R$$

Concrete Behaviour

- Output on a : either b or d is sent
- Input on a : w receives the transmitted value
- Forwarding: whatever w received is output on c

CFA Guess (ρ, κ)

Variable	ρ
a	$\{a\}$
b	$\{b\}$
c	$\{c\}$
d	$\{d\}$
w	$\{b, d\}$

Channel	κ
a	$\{b, d\}$
b	$\emptyset$
c	$\{b, d\}$
d	$\emptyset$

Verification

1. $\bar{a}(b)$: $\rho(b) = \{b\} \subseteq \kappa(a) = \{b, d\}$ ✓
2. $\bar{a}(d)$: $\rho(d) = \{d\} \subseteq \kappa(a) = \{b, d\}$ ✓
3. $a(w)$: $\kappa(a) = \{b, d\} \subseteq \rho(w) = \{b, d\}$ ✓
4. $\bar{c}(w)$: $\rho(w) = \{b, d\} \subseteq \kappa(c) = \{b, d\}$ ✓

Analysis predicts both possible executions (sound over-approximation).

Worked Example 2: Name Mobility

Process:

$$(\nu a)(\bar{a}(b).0 \mid a(x).\bar{c}(x).0) \mid \bar{c}(d).0$$

Analysis

- Channel a is restricted: (νa) creates a fresh name
- Output on a : b is communicated over a
- Input on a : x receives b , then forwards it on c
- Parallel output on c : d is also sent on c

Result

$$\kappa(c) \supseteq \{b, d\}, \quad \rho(x) \supseteq \{b\}$$

The Match Clause in Practice

Consider:

$$a(x).c(y).[x = y].P' \mid \bar{a}(d).Q' \mid \bar{a}(b).Q'' \mid \bar{c}(b).R'$$

- $\rho(x) = \{b, d\}, \rho(y) = \{b\}$
- $\rho(x) \cap \rho(y) = \{b\} \neq \emptyset \rightarrow$ the match can succeed, analyse P'

Now if $\bar{c}(f).R'$ instead of $\bar{c}(b).R'$: - $\rho(x) = \{b, d\}, \rho(y) = \{f\}$ - $\rho(x) \cap \rho(y) = \emptyset \rightarrow$ match can never succeed, P' is unreachable under refined analysis

Theoretical Properties

Subject Reduction (Semantic Correctness)

If $(\rho, \kappa) \models_P P$ and $P \rightarrow Q$ (a reduction step), then $(\rho, \kappa) \models_P Q$.

The analysis result is preserved under computation — it is a **sound over-approximation** of all reachable states.

Preservation under α -renaming

Use **canonical names** (equivalence classes): - $[n]$ is the equivalence class of n - $[x] = x$ for variables - Names may only be substituted within the same equivalence class

Lemma: If $P \equiv_\alpha Q$ then $(\rho, \kappa) \models_P [P] \Leftrightarrow (\rho, \kappa) \models_P [Q]$.

Substitution Lemma

If $(\rho, \kappa) \models_P [P]$ then $(\rho, \kappa) \models_P [P[m/x]]$, provided $[m] \subseteq \rho(x)$.

Existence and Moore Family

The set $\{(\rho, \kappa) \mid (\rho, \kappa) \models_P P\}$ is a **Moore family** (closed under greatest lower bounds). Therefore:

- Every process P has at least one analysis (the trivial one with everything flowing everywhere)
- There exists a **least** (most precise) solution: the intersection of all acceptable pairs

Implementation

- The validation clauses generate a set of logical constraints (set inclusions)
- Constraints can be solved by a specialised constraint solver or by translation to first-order logic
- Complexity is low polynomial

Nature of Imprecision in 0-CFA for π -calculus

The analysis is **context-insensitive** (0-CFA), leading to imprecision:

$$P_1 \triangleq a(x) \mid \bar{a}(b)$$

$$P_2 \triangleq a(x) + \bar{a}(b)$$

$$P_3 \triangleq a(x).\bar{a}(b)$$

In P_1 , x can be bound to b . In P_2 and P_3 , it cannot (no communication possible). However, the 0-CFA estimate gives $\rho(x) \supseteq \{b\}$ in **all three cases** — a safe over-approximation that does not distinguish between reachable and unreachable communications.

Polyadic π -Calculus Extension

Syntax

Allow tuples of values:

$$\begin{aligned} \pi &::= \bar{u}(\vec{v}) \\ &\mid u(\vec{x}) \\ &\mid \tau \end{aligned}$$

Reduction with Arity

[Com] $(\bar{n}(\vec{m}).P + P') \mid (n(\vec{x}).Q + Q') \rightarrow P \mid Q[\vec{m}/\vec{x}]$
provided $|\vec{m}| = |\vec{x}|$ (arity must match).

Extended Abstract Domains

$$\text{Val} = \wp(\text{Const})$$

$$\rho : \text{Var} \rightarrow \wp(\text{Const})$$

$$\kappa : \text{Var} \rightarrow \wp(\text{Const}^*)$$

$$\psi : \wp(\text{Const})$$

- κ now maps to **sequences of constants** (tuples)
- ψ records the set of channels where an **arity mismatch** may occur

Extended Validation (sketch)

$$(\rho, \kappa) \models_A \bar{u}(\vec{v}) \Leftrightarrow \forall n \in \rho(u): \rho(v_1) \times \dots \times \rho(v_k) \subseteq \kappa(n)$$

$$(\rho, \kappa) \models_A u(\vec{x}) \Leftrightarrow \forall n \in \rho(u): \kappa(n) \subseteq \rho(x_1) \times \dots \times \rho(x_k)$$

Arity mismatches are recorded in ψ for error detection.

Complete Formula Handbook

F-01: Collecting Semantics

$$\llbracket c \rrbracket P = \bigcup_{\sigma \in P} \llbracket c \rrbracket \sigma$$

Meaning: The collecting semantics maps a set of input states to the union of all reachable output states.

F-02: Weakest Liberal Precondition

$$\text{wlp}(c, Q) = \{\sigma \mid \llbracket c \rrbracket \{\sigma\} \subseteq Q\}$$

F-03: Weakest Possible Precondition

$$\text{wpp}(c, Q) = \{\sigma \mid \llbracket c \rrbracket \sigma \cap Q \neq \emptyset\}$$

F-04: Hoare Triple Validity

$$\{P\} c \{Q\} \Leftrightarrow \llbracket c \rrbracket P \subseteq Q$$

F-05: Incorrectness Triple Validity

$$[P] c [Q] \Leftrightarrow \llbracket c \rrbracket P \supseteq Q$$

F-06: Galois Connection

$$\alpha(c) \sqsubseteq_A a \Leftrightarrow c \sqsubseteq_C \gamma(a)$$

F-07: Soundness of Abstract Semantics

$$\alpha \circ \llbracket c \rrbracket \sqsubseteq \llbracket c \rrbracket^\# \circ \alpha$$

F-08: Best Abstract Correct Approximation

$$\llbracket c \rrbracket_A^\# = \alpha \circ \llbracket c \rrbracket \circ \gamma$$

F-09: Completeness Equation

$$\alpha \circ \llbracket c \rrbracket = \alpha \circ \llbracket c \rrbracket \circ \gamma \circ \alpha$$

Exam Preparation Guide

Recurring Exercise Patterns

1. **Prove a Hoare triple** using HL inference rules — find the loop invariant
2. **Prove an incorrectness triple** using IL rules — find the right under-approximation
3. **Galois connection** — define α and γ for a given abstraction
4. **Abstract semantics** — define $\llbracket c \rrbracket^\#$ for a given abstract domain
5. **Fixpoint computation** — compute the least fixpoint for a loop in an abstract domain
6. **CFA constraints** — generate constraints for a functional program

Common Proof Strategies

- **Induction on derivation trees** for proving soundness
- **Fixpoint induction** (Kleene) for loop semantics
- **Structural induction** for semantic properties
- **Coinduction** for non-termination/infinite behaviors

Common Lattice Constructions

1. **Product lattice:** $L_1 \times L_2$ with pointwise order
2. **Flat lattice:** $L^\#$ with $\perp < a < \top$
3. **Interval lattice:** $\{\llbracket l, u \rrbracket \mid l \leq u, l \in \mathbb{Z} \cup \{-\infty\}, u \in \mathbb{Z} \cup \{+\infty\}\}$
4. **Sign lattice:** $\{\emptyset, < 0, = 0, > 0, \leq 0, \geq 0, \mathbb{Z}\}$ ordered by subset inclusion

Frequently Tested Concepts

- wlp vs wpp and their properties
 - HL vs IL duality
 - Soundness and completeness
 - Galois connections and insertions
 - Abstract domain design
 - Widening for fixpoint termination
 - 0-CFA for functional programs
 - Flow logic formulation of CFA
-

Final Cheat Sheet (Expanded)

1. Denotational Semantics

Concept	Definition
State	$\Sigma \triangleq \{\sigma: X \rightarrow \mathbb{Z}\}$
Collecting semantics	$\llbracket c \rrbracket: \wp(\Sigma) \rightarrow \wp(\Sigma), \llbracket c \rrbracket P = \bigcup_{\sigma \in P} \llbracket c \rrbracket \sigma$
Skip	$\llbracket \text{skip} \rrbracket P = P$
Assignment	$\llbracket x := a \rrbracket P = \{\sigma[n/x] \mid \sigma \in P, n = \llbracket a \rrbracket \sigma\}$
Sequence	$\llbracket c_1; c_2 \rrbracket P = \llbracket c_2 \rrbracket (\llbracket c_1 \rrbracket P)$
Choice	$\llbracket c_1 + c_2 \rrbracket P = \llbracket c_1 \rrbracket P \cup \llbracket c_2 \rrbracket P$
Iteration	$\llbracket c^* \rrbracket P = \bigcup_{k=0}^{\infty} \llbracket c \rrbracket^k P$
Guard	$\llbracket b? \rrbracket P = P \wedge b$
Encoding if	$\text{if } b \text{ then } c_1 \text{ else } c_2 \equiv (b?; c_1) + (\neg b?; c_2)$
Encoding while	$\text{while } b \text{ do } c \equiv (b?; c)^*; \neg b?$
wlp	$\text{wlp}(c, Q) = \{\sigma \mid \llbracket c \rrbracket \{\sigma\} \subseteq Q\}$
wpp	$\text{wpp}(c, Q) = \{\sigma \mid \llbracket c \rrbracket \sigma \cap Q \neq \emptyset\}$

2. Program Logics — The Four Quadrants

Logic	Triple	Direction	Approximation	Validity Condition
Hoare Logic (HL)	$\{P\} c \{Q\}$	Forward	Over ($\subseteq$)	$\llbracket c \rrbracket P \subseteq Q$
Incorrectness Logic (IL)	$[P] c [Q]$	Forward	Under ($\supseteq$)	$\llbracket c \rrbracket P \supseteq Q$
Necessary Condition (NC)	$\langle P \rangle c \langle Q \rangle$	Backward	Under	$P \subseteq \llbracket c \rrbracket^{\text{op}} Q$

Logic	Triple	Direction	Approximation	Validity Condition
Sufficient Incorrectness (SIL)	$(P) \ c \ (Q)$	Backward	Over	$P \supseteq \llbracket c \rrbracket^{\text{op}} Q$

First-order Characterizations

- HL: $\forall \sigma \in P. \forall \delta \in \llbracket c \rrbracket \sigma. \delta \in Q$
- IL: $\forall \delta \in Q. \exists \sigma \in P. \delta \in \llbracket c \rrbracket \sigma$

3. Hoare Logic Rules (Partial Correctness)

Rule	Formula
Skip	$\{P\} \text{ skip } \{P\}$
Assignment (Hoare)	$\{Q[a/x]\} \ \bar{x} := a \ \{Q\}$
Assignment (Floyd)	$\{P\} \ x := a \ \{\exists x'. P[\bar{x}'/x] \wedge x = a[x'/x]\}$
Sequence	$\frac{\{P\} \ c_1 \ \{R\} \quad \{R\} \ c_2 \ \{Q\}}{\{P\} \ c_1; c_2 \ \{Q\}}$
Conditional	$\frac{\{P \wedge b\} \ c_1 \ \{Q\} \quad \{P \wedge \neg b\} \ c_2 \ \{Q\}}{\{P\} \ \text{if } b \text{ then } c_1 \ \text{else } c_2 \ \{Q\}}$
While	$\frac{\{P \wedge b\} \ c \ \{P\}}{\{P\} \ \text{while } b \text{ do } c \ \{P \wedge \neg b\}}$
Consequence	$\frac{P \Rightarrow P' \quad \{P'\} \ c \ \{Q'\} \quad Q' \Rightarrow Q}{\{P\} \ c \ \{Q\}}$
While (total)	$\frac{\{P \wedge b\} \ c \ \{P\} \quad \{P \wedge b \wedge t = z\} \ c \ \{t < z\} \quad P \Rightarrow t \geq 0}{\{P\} \ \text{while } b \text{ do } c \ \{P \wedge \neg b\}}$

4. Incorrectness Logic Rules

Rule	Formula
Skip	$[P] \text{ skip } [P]$
Assignment (Floyd)	$[P] \ x := a \ [\exists x'. P[\bar{x}'/x] \wedge x = a[x'/x]]$
Assume	$[P] \ b? \ [\bar{P} \wedge b]$
Error	$[P] \ \text{error}() \ [\text{false}]$
Nondet	$[P] \ x := \text{nondet}() \ [\exists x. P]$
Sequence	$\frac{[P] \ c_1 \ [R] \quad [R] \ c_2 \ [Q]}{[P] \ c_1; c_2 \ [Q]}$
Choice	$\frac{[P] \ c_1 \ [Q_1] \quad [P] \ c_2 \ [Q_2]}{[P] \ c_1 + c_2 \ [Q_1 \vee Q_2]}$

Rule	Formula
Consequence (IL)	$\frac{P' \Rightarrow P \quad [P'] \text{ c } [Q'] \quad Q \Rightarrow Q'}{[P] \text{ c } [Q]}$
Disjunction	$\frac{[P_1] \text{ c } [Q_1] \quad [P_2] \text{ c } [Q_2]}{[P_1 \vee P_2] \text{ c } [Q_1 \vee Q_2]}$
Frame (IL)	$\frac{[P] \text{ c } [Q]}{[P \wedge R] \text{ c } [Q \wedge R]} \text{ (modifies } \cap \text{ FV}(R) = \emptyset)$
Iteration	$\frac{\forall n \geq 0. [P_n] \text{ c } [P_{n+1}]}{[P_0] \text{ c }^* [\exists k. P_k]}$

5. Separation Logic

Heap Model

- State: (s, h) where $s: \text{Var} \rightarrow \text{Val}$, $h: \text{Loc} \rightarrow \text{Val}$
- $h_1 \cdot h_2$: union of disjoint heaps (domains are disjoint)
- $s, h \models \text{emp}$ iff $\text{dom}(h) = \emptyset$
- $s, h \models x \mapsto v$ iff $h = \{s(x) \mapsto s(v)\}$
- $s, h \models P \star Q$ iff $\exists h_1, h_2: h = h_1 \cdot h_2, s, h_1 \models P, s, h_2 \models Q$
- $s, h \models P - * Q$ iff $\forall h' \perp h: s, h' \models P \Rightarrow s, h \cdot h' \models Q$

Key SL Rules

Rule	Formula
Frame	$\frac{\{P\} \text{ c } \{Q\}}{\{P \star R\} \text{ c } \{Q \star R\}} \text{ (no mod. var in } R)$
Alloc	$\{\text{emp}\} x := \text{alloc}() \{x \mapsto _ \}$
Read	$\{x \mapsto v\} y := [x] \{x \mapsto v \wedge y = v\}$
Write	$\{x \mapsto _ \} [x] := y \{x \mapsto y\}$
Free	$\{x \mapsto _ \} \text{free}(x) \{\text{emp}\}$

List Predicates

- $ls(a_1, a_2) \triangleq (a_1 = a_2 \wedge \text{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell. a_1 \mapsto \ell \star ls(\ell, a_2))$
- $\text{list}(a) \triangleq ls(a, \text{nil})$

ISL Heap Rules

Rule	Formula
Read	$[v \mapsto w \star P] x := [v] [v \mapsto w \star P \wedge x = w]$
Write	$[v \mapsto _ \star P] [v] := y [v \mapsto y \star P]$
Alloc	$[\text{emp}] x := \text{alloc}() [x \mapsto _]$

Rule	Formula
Free	$[v \mapsto _]\text{ free}(v) [\text{emp}]$

6. Abstract Interpretation

Galois Connections

Property	Formula
GC definition	$\alpha(c) \sqsubseteq_A a \Leftrightarrow c \sqsubseteq_C \gamma(a)$
α preserves	Least upper bounds ($\sqcup$)
γ preserves	Greatest lower bounds ($\sqcap$)
GI condition	α surjective, γ injective, $\alpha \circ \gamma = \text{id}$
Closure operator	$\gamma \circ \alpha$ is extensive, monotone, idempotent
Expressible elements	$c = \gamma(\alpha(c))$

Abstract Semantics Soundness

Concept	Formula
Soundness	$\alpha(\llbracket c \rrbracket P) \sqsubseteq \llbracket c \rrbracket^\# (\alpha(P))$
Equivalent	$\llbracket c \rrbracket \gamma(a) \subseteq \gamma(\llbracket c \rrbracket^\# a)$
BCA	$\llbracket c \rrbracket_A^\# = \alpha \circ \llbracket c \rrbracket \circ \gamma$
Completeness	$\alpha \circ \llbracket c \rrbracket = \llbracket c \rrbracket^\# \circ \alpha$

Local Completeness Logic (LCL)

Concept	Formula
Closure A	$A = \gamma \circ \alpha$
Global completeness	$\forall P. A(\llbracket c \rrbracket P) = \llbracket c \rrbracket^\# A(P)$
Local completeness	$\mathbb{C}_P(e): A(\llbracket e \rrbracket P) = A(\llbracket e \rrbracket A(P))$
LCL triple	$\vdash_A [P] c [Q] \text{ with } Q \subseteq \llbracket c \rrbracket P \subseteq A(Q)$
Verification theorem	LCL derivation + local completeness $\Rightarrow$ global soundness

Abstract Domains

Domain	Elements	ACC?	Needs Widening?
Sign	$\{\perp, -, 0, +, \top\}$	Yes	No
Constant	$\{\perp, c, \top\}$ for $c \in \mathbb{Z}$	No	Yes
Interval	$\{\llbracket l, h \rrbracket \mid l \leq h\} \cup \{\perp\}$	No	Yes (∇)
Congruence	$a\mathbb{Z} + b$	Yes	No

Domain	Elements	ACC?	Needs Widening?
Octagon	$\pm x \pm y \leq c$	No	Yes
Polyhedron	$\sum a_i x_i \leq d$	No	Yes

Interval Operations

- $[l_1, h_1] + [l_2, h_2] = [l_1 + l_2, h_1 + h_2]$
- $[l_1, h_1] \times [l_2, h_2] = [\min(l_1 l_2, l_1 h_2, h_1 l_2, h_1 h_2), \max(\dots)]$
- $[a, b] \nabla [c, d] = [\text{if } c < a \text{ then } -\infty \text{ else } a, \text{if } d > b \text{ then } +\infty \text{ else } b]$
- $[a, b] \Delta [c, d] = [\text{if } a = -\infty \text{ then } c \text{ else } a, \text{if } b = +\infty \text{ then } d \text{ else } b]$

7. Control Flow Analysis (0-CFA for FUN)

Abstract Domains

- $\hat{C}: \text{Lab} \rightarrow \wp(\text{Term})$ — cache: which function abstractions reach each program point
- $\hat{\rho}: \text{Var} \rightarrow \wp(\text{Term})$ — environment: which abstractions each variable may bind

Acceptability Clauses $(\hat{C}, \hat{\rho}) \models e$

Expression	Clause
c^ℓ	$\emptyset \subseteq \hat{C}(\ell)$
x^ℓ	$\hat{\rho}(x) \subseteq \hat{C}(\ell)$
$(\text{fn } x \Rightarrow t)^\ell$	$\{\text{fn } x \Rightarrow t\} \subseteq \hat{C}(\ell)$
$(t_1 \ t_2)^\ell$	$(\hat{C}, \hat{\rho}) \models t_1 \wedge (\hat{C}, \hat{\rho}) \models t_2 \wedge \forall \text{fn } x \Rightarrow t_0 \in \hat{C}(\ell_1): \hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)$
$(\text{let } x = t_1 \text{ in } t_2)^\ell$	$(\hat{C}, \hat{\rho}) \models t_1 \wedge \hat{\rho}(x) = \hat{C}(\ell_1) \wedge (\hat{C}, \hat{\rho}) \models t_2$

Constraint Generation $\mathcal{C}^*[[e]]$

Expression	Constraints
c^ℓ	$\emptyset$
x^ℓ	$r(x) \subseteq \mathcal{C}(\ell)$
$(\text{fn } x \Rightarrow t)^\ell$	$\{\text{fn } x \Rightarrow t\} \subseteq \mathcal{C}(\ell) \cup \mathcal{C}^*[[t]]$
$(t_1 \ t_2)^\ell$	$\mathcal{C}^*[[t_1]] \cup \mathcal{C}^*[[t_2]] \cup \{\{\text{fn } x \Rightarrow t_0\} \subseteq \mathcal{C}(\ell_1) \Rightarrow \mathcal{C}(\ell_2) \subseteq r(x)\} \cup \{\cdot\}$

Key Properties

- **Order:** $(\hat{C}_1, \hat{\rho}_1) \sqsubseteq (\hat{C}_2, \hat{\rho}_2)$ iff $\forall \ell. \hat{C}_1(\ell) \subseteq \hat{C}_2(\ell)$ and $\forall x. \hat{\rho}_1(x) \subseteq \hat{\rho}_2(x)$
- **Moore family:** the set of acceptable analyses is closed under $\sqcap \rightarrow$ least (most precise) solution exists

- **0-CFA**: monovariant (one value per program point)
- **k-CFA**: context-sensitive with call strings of length k

CPA (Cartesian Product Algorithm)

- Contexts: $\Delta = \text{Term}^m$ (tuples of arguments)
- $\hat{\rho}: (\text{Var} \times \Delta) \rightarrow \wp(\text{Term})$
- $\hat{C}: (\text{Lab} \times \Delta) \rightarrow \wp(\text{Term})$

8. 0-CFA for π -Calculus

Abstract Domains

- $\rho: \text{Var} \rightarrow \wp(\text{Const})$ — which constants each variable may bind
- $\kappa: \text{Var} \rightarrow \wp(\text{Const})$ — which constants may be communicated over each channel
- Extension: $\rho(n) = \{n\}$ for constants n

Validation Clauses

Construct	Clause
[out] $\bar{u}(v)$	$\forall n \in \rho(u): \rho(v) \subseteq \kappa(n)$
[in] $u(x)$	$\forall n \in \rho(u): \kappa(n) \subseteq \rho(x)$
[tau] τ	Always true
[pref] $\pi.P$	$(\rho, \kappa) \models_A \pi \wedge (\rho, \kappa) \models_P P$
[par] $P_1 \mid P_2$	$(\rho, \kappa) \models_P P_1 \wedge (\rho, \kappa) \models_P P_2$
[sum] $\sum_i \pi_i . P_i$	$\forall i: (\rho, \kappa) \models_A \pi_i \wedge (\rho, \kappa) \models_P P_i$
[match] $[x = y]P$	$(\rho, \kappa) \models_P P$ (or $\rho(x) \cap \rho(y) \neq \emptyset \Rightarrow (\rho, \kappa) \models_P P$)
[res] $(\nu n)P$	$(\rho, \kappa) \models_P P$
[rep] $!P$	$(\rho, \kappa) \models_P P$

Theoretical Properties

- **Subject reduction**: $P \rightarrow Q$ and $(\rho, \kappa) \models_P P \Rightarrow (\rho, \kappa) \models_P Q$
- **Substitution lemma**: $[m] \subseteq \rho(x) \Rightarrow (\rho, \kappa) \models_P [P[m/x]]$
- **Moore family**: $\{(\rho, \kappa) \mid (\rho, \kappa) \models_P P\}$ is closed under $\sqcap \rightarrow$ least solution exists

9. Common Exam Tricks

Pattern	Technique
Prove HL triple	Use consequence to weaken postcondition / strengthen precondition

Pattern	Technique
Prove IL triple	Show every poststate has a prestate witness
Find loop invariant	Look for preserved relationship (e.g., $x = n!$ in factorial)
Find loop variant	Decreasing natural-number expression (e.g., $n - i$ in factorial)
Construct Galois connection	Define γ first (most natural), then α as best approximation
Determine completeness	Check if $\alpha \circ F = \alpha \circ F \circ \gamma \circ \alpha$ holds for the given P
Generate CFA constraints	Follow syntax-directed rules, track labels carefully
Predict π -calculus communications	Output: $\rho(v) \subseteq \kappa(u)$; Input: $\kappa(u) \subseteq \rho(x)$
Check Moore family	Verify closure under $\sqcap$ (intersection of any set of solutions is a solution)
Design widening	Force unstable bounds to $\pm\infty$

10. Key Notation Summary

Symbol	Meaning
$\llbracket c \rrbracket$	Denotational semantics of command c
$\{P\} c \{Q\}$	Hoare triple (over, forward)
$[P] c [Q]$	IL triple (under, forward)
$\langle P \rangle c \langle Q \rangle$	NC triple (under, backward)
$(P) c (Q)$	SIL triple (over, backward)
α, γ	Abstraction, concretization
$\sqsubseteq, \sqcup, \sqcap$	Lattice ordering, join, meet
$\perp, \top$	Bottom, top of lattice
∇, Δ	Widening, narrowing
$\hat{C}, \hat{\rho}$	CFA cache, CFA environment
ρ, κ	π -calculus abstract env, channel env
$\mathcal{C}(\ell), r(x)$	CFA constraint variables
$A = \gamma \circ \alpha$	Closure operator
$(\hat{C}, \hat{\rho}) \models e$	CFA acceptability
$(\rho, \kappa) \models_P P$	π -calculus CFA acceptability

Algorithm Extraction

A-01: Kleene Fixpoint Iteration

Purpose

Compute the least fixpoint of a monotone function F on a complete lattice, used to compute loop semantics in both concrete and abstract settings.

Input

- A monotone function $F: L \rightarrow L$ on a complete lattice $(L, \sqsubseteq, \perp, \top, \sqcup, \sqcap)$

Output

- $\text{lfp}(F)$, the least fixpoint of F

Pseudocode

```
x ← ⊥
repeat
  x ← F(x)
until x is a fixpoint (F(x) = x)
return x
```

Complexity

- Termination depends on the height of the lattice L
- If L has finite height h , at most h iterations
- If L has infinite height (e.g., intervals), may not terminate without widening

Correctness Intuition

Kleene's fixpoint theorem: for a continuous function on a CPO, the least fixpoint equals $\bigsqcup_{n \geq 0} F^n(\perp)$.

Common Mistakes

- Assuming termination on infinite-height lattices without widening
- Confusing least fixpoint with greatest fixpoint
- Forgetting monotonicity requirement

Exam Relevance

Fundamental for understanding loop analysis in abstract interpretation. Used in Lectures 12, 15.

A-02: Abstract Fixpoint with Widening

Purpose

Compute a sound over-approximation of the least fixpoint when the abstract lattice has infinite height (e.g., intervals).

Input

- A monotone function $F^\# : A \rightarrow A$ on an abstract domain A
- A widening operator $\nabla : A \times A \rightarrow A$

Output

- A post-fixpoint x such that $F^\#(x) \sqsubseteq x$ (sound approximation of $\text{lfp}(F)$)

Pseudocode

```
x ← ⊥
repeat
  x ← x ∇ F#(x)
until F#(x) ⊆ x
return x
```

Complexity

Widening guarantees termination in finite time regardless of lattice height.

Correctness Intuition

If x is a post-fixpoint ($F^\#(x) \sqsubseteq x$), then $\text{lfp}(F) \sqsubseteq x$ by soundness.

Common Mistakes

- Widening too early, causing precision loss
- Using $\sqcup$ instead of ∇ (no termination guarantee)
- Forgetting that widening must be used at loop heads only

Exam Relevance

Central for interval analysis. Frequently examined in combination with narrowing for precision recovery.

A-03: Worklist Algorithm for Abstract Interpretation

Purpose

Efficiently compute the least fixpoint of a system of abstract semantic equations over a control flow graph.

Input

- CFG with program points $\ell_1, \dots, \ell_n$
- Initial abstract state a_0 at entry point
- Abstract transfer functions $F_\ell^\#$ for each edge

Output

- Abstract invariant $I(\ell)$ for each program point ℓ

Pseudocode

```
for each  $\ell$ :  $I(\ell) \leftarrow \perp$ 
 $I(\ell_{\text{entry}}) \leftarrow a_0$ 
 $W \leftarrow \{\ell_{\text{entry}}\}$ 
while  $W \neq \emptyset$ :
     $\ell \leftarrow \text{remove from } W$ 
    for each successor  $\ell'$  of  $\ell$ :
         $\text{new} \leftarrow I(\ell) \sqcup F_{\ell \rightarrow \ell'}^\#(I(\ell))$ 
        if  $\text{new} \neq I(\ell')$ :
             $I(\ell') \leftarrow \text{new}$ 
             $W \leftarrow W \cup \{\ell'\}$ 
return  $I$ 
```

Complexity

- Each edge processed at most h times where h is lattice height
- With widening: edges processed at most once per widening point

Correctness Intuition

Join with existing values ensures monotonic increase; termination by lattice ACC or widening.

Common Mistakes

- Forgetting to join new and old values (instead of just assigning)
- Missing widening at loop heads
- Lattice height assumed finite when it's not

Exam Relevance

Standard algorithm for implementing static analyses. Likely asked in exam.

A-04: 0-CFA Constraint Generation

Purpose

Generate a system of constraints whose least solution gives the 0-CFA of a functional program (monovariant CFA).

Input

- A labeled expression e with labels $\ell \in \text{Lab}$
- Abstraction: set of function abstractions Term

Output

- A set $\mathcal{C}^*[[e]]$ of constraints and conditional constraints over variables $\mathcal{C}(\ell)$ and $r(x)$
- Least solution $(\hat{\mathcal{C}}, \hat{\rho})$ is the most precise 0-CFA

Constraint Generation Rules

Constants: $\mathcal{C}^*[[c^\ell]] = \emptyset$

Variables: $\mathcal{C}^*[[x^\ell]] = \{r(x) \subseteq \mathcal{C}(\ell)\}$

Functions: $\mathcal{C}^*[(\text{fn } x \Rightarrow e_0)^\ell] = \{\{\text{fn } x \Rightarrow e_0\} \subseteq \mathcal{C}(\ell)\} \cup \mathcal{C}^*[[e_0]]$

Application: $\mathcal{C}^*[[e_1^{\ell_1} e_2^{\ell_2}]^\ell] = \mathcal{C}^*[[e_1]] \cup \mathcal{C}^*[[e_2]] \cup$
 $\{\{\text{fn } x \Rightarrow e_0\} \subseteq \mathcal{C}(\ell_1) \Rightarrow \mathcal{C}(\ell_2) \subseteq r(x) \mid \forall \text{fn } x \Rightarrow e_0 \in \text{Term}\}$
 $\cup \{\{\text{fn } x \Rightarrow e_0\} \subseteq \mathcal{C}(\ell_1) \Rightarrow \mathcal{C}(\ell_0) \subseteq \mathcal{C}(\ell) \mid \forall \text{fn } x \Rightarrow e_0 \in \text{Term}\}$

Let:

$\mathcal{C}^*[(\text{let } x = t_1^{\ell_1} \text{ in } t_2^{\ell_2})^\ell] = \mathcal{C}^*[[t_1]] \cup \mathcal{C}^*[[t_2]] \cup \{\mathcal{C}(\ell_1) \subseteq r(x), \mathcal{C}(\ell_2) \subseteq \mathcal{C}(\ell)\}$

Complexity

- $O(|e|^2)$ constraints generated
- Least solution computable via fixpoint in $O(|e|^3)$

Correctness Intuition

Constraints encode the acceptability relation $(\hat{\mathcal{C}}, \hat{\rho}) \models e$; least solution is most precise analysis.

Common Mistakes

- Forgetting conditional constraints for applications
- Confusing $\subseteq$ direction
- Not handling all function abstractions in Term

Exam Relevance

High — 0-CFA is a staple exam topic. Exercises require constraint generation and solution.

A-05: Interval Widening

Purpose

Ensure termination of fixpoint iteration for the interval domain by forcing unstable bounds to $\pm\infty$.

Input

- Two intervals $[a, b], [c, d]$ with $a, b, c, d \in \mathbb{Z} \cup \{-\infty, +\infty\}$

Output

- An interval $[a, b] \nabla [c, d]$ that over-approximates both

Formula

$$[a, b] \nabla [c, d] = \begin{cases} [-\infty, b] & \text{if } c < a \\ [a, +\infty] & \text{if } d > b \\ [a, b] & \text{otherwise} \end{cases}$$

Complexity

$O(1)$

Correctness Intuition

Once a bound moves in a direction, it is pushed to infinity, so it cannot change again. Only finitely many changes per bound.

Common Mistakes

- Widening from $\perp$ gives $[-\infty, +\infty]$ immediately (too imprecise)
- Widening after stabilization is unnecessary

- Forgetting that widening loses precision permanently

Exam Relevance

High — interval widening is a classic exam question. Often combined with narrowing.

A-06: Syntax-Directed CFA ($\models_s$)

Purpose

Compute CFA using a syntax-directed system that analyzes function bodies once at definition time.

Input

- Labeled expression e

Output

- $\mathcal{C}^*[[e]]$, a set of constraints

Key Difference from Acceptability

The syntax-directed relation $\models_s$ analyzes function bodies once regardless of call count. This means: - Functions are analyzed even if never called - No “call-context” sensitivity (0-CFA) - Better for modular analysis

Rules

Same as acceptability but without runtime context: function body constraints are generated at definition time.

Complexity

Linear in program size.

A-07: k -CFA (Call String Approach)

Purpose

Refine 0-CFA by distinguishing calling contexts up to depth k .

Input

- Expression e , bound k on call string length

Output

- Context-sensitive cache $\hat{C}(\ell, \delta)$ and environment $\hat{\rho}(x, \delta)$ where δ is a call string of length $\leq k$

Key Change

- Abstract values replaced by $(\hat{C}, \hat{\rho}, \delta)$ where δ tracks the call site history
- $k = 0$: monovariant (0-CFA)
- $k = 1$: call-site sensitivity
- $k = 2$: two-level call stack

Complexity

- $O(|e|^{k+1})$ — exponential in k
- $k = 1$ is common practical choice

Correctness Intuition

Different call sites create different instances of formal parameters, removing spurious flows from merged contexts.

Common Mistakes

- Thinking k -CFA is always more precise (it is, but at cost)
- Forgetting that k -CFA still over-approximates

Exam Relevance

Exercise E-03 asks to compare 0-CFA and 1-CFA results.

Complete Exercise Collection

Source

All exercises from the official exercise PDFs and their extracted text versions.

Easy

Exercise E-01: Guard Axiom Across Logics

Source: ProgramAnalysis_exercises_a.txt (Ex. 1) **Lecture:** Lectures 03–08 (HL, IL, NC, SIL) **Page:** 1 **Difficulty:** Easy **Tags:** hoare logic, incorrectness logic, necessary condition logic, sufficient incorrectness logic, boolean guards

Statement

Let us consider the following axiom for atomic boolean guards $b?$, where the parentheses $\langle \cdot | \cdot \rangle \in \{\{\cdot\}, [\cdot], (\cdot), \langle \cdot \rangle\}$ are used to denote a generic program logic triple of either Hoare logic, IL, NC or SIL.

$$\langle \langle P | \rangle \rangle b? \langle \langle P \vee b | \rangle \rangle$$

Explain for which logics (HL, IL, NC, SIL) the axiom is sound (for any precondition P and boolean guard b): - if the axiom is sound, prove its validity; - if it is not sound, first provide a concrete counterexample and then say under which conditions on P and b the axiom would become sound.

Official Solution

Hoare Logic. A Hoare triple $\{P\} c \{Q\}$ is sound when every terminating execution from a state satisfying P ends in a state satisfying Q . Since executing $b?$ does not modify the state, every reachable state satisfies both P and b . In fact:

$$\llbracket b? \rrbracket P = (P \wedge b) \subseteq (P \vee b).$$

Hence the axiom is **sound** for Hoare Logic.

Incorrectness Logic. A valid IL triple $[P] c [Q]$ requires $Q \subseteq \llbracket c \rrbracket P$. For the proposed axiom we would need:

$$P \vee b \subseteq \llbracket b? \rrbracket P = P \wedge b.$$

This is false in general. Counterexample: $P = \text{true}$, $b = \text{false}$. Then $P \vee b = \text{true}$, $P \wedge b = \text{false}$. The axiom becomes sound iff $P \vee b = P \wedge b$, i.e., iff $P = b$.

Necessary Condition Logic. NC requires $\llbracket c \rrbracket^{\text{op}} Q \subseteq P$. Since $\llbracket b? \rrbracket^{\text{op}} Q = Q \wedge b$:

$$\llbracket b? \rrbracket^{\text{op}} (P \vee b) = (P \vee b) \wedge b = b.$$

The axiom is sound for NC iff $b \subseteq P$. Counterexample: $P = \text{false}$, $b = \text{true}$.

Sufficient Incorrectness Logic. SIL requires an under-approximation condition: $P \subseteq \llbracket c \rrbracket^{\text{op}} Q$. Thus:

$$P \subseteq \llbracket b? \rrbracket^{\text{op}} (P \vee b) = (P \vee b) \wedge b = b.$$

This is false in general. Counterexample: $P = \text{true}$, $b = \text{false}$.

Key Concepts

- HL triple validity: $\llbracket c \rrbracket P \subseteq Q$
- IL triple validity: $\llbracket c \rrbracket P \supseteq Q$
- NC triple validity: $\llbracket c \rrbracket^{\text{op}} Q \subseteq P$
- SIL triple validity: $P \subseteq \llbracket c \rrbracket^{\text{op}} Q$
- Boolean guard semantics: $\llbracket b? \rrbracket P = P \wedge b$

Typical Exam Insight

Tests understanding of the fundamental difference between over-approximation (HL) and under-approximation (IL/NC/SIL) logics.

Medium

Exercise E-02: Galois Connection Design for Abstract Domain

Source: ProgramAnalysis_exercises_a.txt (Ex. 2) **Lecture:** Lectures 12–14 (Galois Connections, Abstract Domains) **Page:** 1 **Difficulty:** Medium **Tags:** galois connection, abstraction map, concretization map, best correct approximation, completeness, sign domain

Statement

Let us consider the usual concrete domain $(\wp(\mathbb{Z}), \subseteq)$ of sets of integers ordered by inclusion and the abstract domain $A = \{\perp, = 0, \leq 0, > 10, > 0, \top\}$ with the obvious concretization function.

1. Define the GC maps $\gamma: A \rightarrow \wp(\mathbb{Z})$ and $\alpha: \wp(\mathbb{Z}) \rightarrow A$.
2. Draw the Hasse diagram of A .
3. Define the bca $\cdot \times_A \cdot$ for the product of integers $\cdot \times \cdot$.
4. Show that $\cdot \times_A \cdot$ is not complete by exhibiting a counterexample.

Official Solution

1. Galois connection maps:

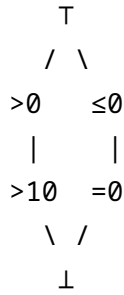
Concretization:

$$\begin{aligned} \gamma(\perp) &= \emptyset, & \gamma(= 0) &= \{0\}, & \gamma(\leq 0) &= \{z \in \mathbb{Z} \mid z \leq 0\}, \\ \gamma(> 10) &= \{z \in \mathbb{Z} \mid z > 10\}, & \gamma(> 0) &= \{z \in \mathbb{Z} \mid z > 0\}, & \gamma(\top) &= \mathbb{Z}. \end{aligned}$$

Abstraction:

$$\alpha(X) = \begin{cases} \perp & \text{if } X = \emptyset \\ = 0 & \text{if } X = \{0\} \\ \leq 0 & \text{else if } X \subseteq \{z \mid z \leq 0\} \\ > 10 & \text{else if } X \subseteq \{z \mid z > 10\} \\ > 0 & \text{else if } X \subseteq \{z \mid z > 0\} \\ \top & \text{otherwise} \end{cases}$$

2. Hasse diagram:



3. Best correct approximation of multiplication:

$$a \times_A b = \alpha(\{x \cdot y \mid x \in \gamma(a), y \in \gamma(b)\})$$

$\times_A$	$\perp$	$= 0$	≤ 0	> 10	> 0	$\top$
$\perp$	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
$= 0$	$\perp$	$= 0$	$= 0$	$= 0$	$= 0$	$= 0$
≤ 0	$\perp$	$= 0$	$\top$	≤ 0	≤ 0	$\top$
> 10	$\perp$	$= 0$	≤ 0	> 10	> 10	$\top$
> 0	$\perp$	$= 0$	≤ 0	> 10	> 0	$\top$
$\top$	$\perp$	$= 0$	$\top$	$\top$	$\top$	$\top$

4. Non-completeness:

Take $X = \{-1\}$, $Y = \{-1\}$. Then $\alpha(X) = \leq 0$, $\alpha(Y) = \leq 0$. Concrete multiplication gives $X \times Y = \{1\}$, so $\alpha(X \times Y) = (> 0)$. However, $\leq 0 \times_A \leq 0 = \top$ because the products may be 0 or positive. Therefore $\alpha(X \times Y) \neq \alpha(X) \times_A \alpha(Y)$, proving incompleteness.

Key Concepts

- Galois connection definition: $\alpha(c) \sqsubseteq_A a \Leftrightarrow c \subseteq \gamma(a)$
- BCA formula: $F^A = \alpha \circ F \circ \gamma$
- Completeness equation: $\alpha \circ F = F^\# \circ \alpha$
- Formula F-06, F-07, F-08, F-09

Typical Exam Insight

Tests understanding of Galois connection construction and the distinction between soundness and completeness.

Exercise E-03: CFA for Higher-Order Functions

Source: ProgramAnalysis_exercises_a.txt (Ex. 3) **Lecture:** Lectures 17–22 (CFA)

Page: 2 **Difficulty:** Medium **Tags:** CFA, 0-CFA, 1-CFA, constraint generation, flow logic, acceptability

Statement

Consider the following expression e :

```
let apply = fn f => f 2
in let inc = fn x => x + 4
in let dbl = fn y => y * 2
in (apply inc) - (apply dbl)
```

1. Add labels to all subexpressions and function abstractions.
2. Guess a possible analysis result for Pure 0-CFA and verify it using the corresponding acceptability relation $\models$. In particular, comment on the abstract values associated with the two calls to `apply`.
3. Consider the following additional abstract flow: $\text{fn } w \Rightarrow w \in \hat{C}(\text{Label}(f))$. Does the acceptability relation $\models$ still hold? What does this say about the difference between soundness and precision?
4. Discuss how the analysis changes using the syntax-directed version of the acceptability relation $\models_s$.
5. Extract the corresponding system of constraints.
6. Explain how the result changes under 1-CFA. Which spurious flows disappear, and why?

Official Solution (Summary)

1. Labeled expression:

$$e \triangleq (\text{let } \text{apply} = (\text{fn } f \Rightarrow (f^1 \ 2^2)^3)^4 \\ \text{in } (\text{let } \text{inc} = (\text{fn } x \Rightarrow (x^5 + 4^6)^7)^8 \\ \text{in } (\text{let } \text{dbl} = (\text{fn } y \Rightarrow (y^9 * 2^{10})^{11})^{12} \\ \text{in } ((\text{apply}^{13} \ \text{inc}^{14})^{15} - (\text{apply}^{16} \ \text{dbl}^{17})^{18})^{19})^{20})^{21})^{22}$$

2. 0-CFA result: - $\hat{C}(4) = \{\text{fn } f \Rightarrow (f^1 \ 2^2)^3\}$ - $\hat{C}(8) = \{\text{fn } x \Rightarrow (x^5 + 4^6)^7\}$ -
 $\hat{C}(12) = \{\text{fn } y \Rightarrow (y^9 * 2^{10})^{11}\}$ - $\hat{C}(13) = \hat{C}(16) = \{\text{fn } f \Rightarrow (f^1 \ 2^2)^3\}$ -
 $\hat{C}(14) = \{\text{fn } x \Rightarrow (x^5 + 4^6)^7\}$, $\hat{C}(17) = \{\text{fn } y \Rightarrow (y^9 * 2^{10})^{11}\}$ -
 $\hat{C}(1) = \{\text{fn } x \Rightarrow (x^5 + 4^6)^7, \text{fn } y \Rightarrow (y^9 * 2^{10})^{11}\}$

Key observation: 0-CFA merges different calling contexts — $\hat{C}(14) \subseteq \hat{\rho}(f)$ and $\hat{C}(17) \subseteq \hat{\rho}(f)$ both hold, so f receives both `inc` and `dbl`.

3. Extra flow: Adding $\text{fn } w \Rightarrow w \in \hat{C}(1)$ is still sound (over-approximation) but less precise. This illustrates that acceptability expresses soundness, not optimal precision.

4. Syntax-directed $\models_s$: Analyzes function bodies once at definition time, not at each application. No runtime context.

5. Constraint system: Generated compositionally. Key constraints propagate from calls to f 's body.

6. 1-CFA: Distinguishes the two call sites with call strings $\delta_1 = \langle 15 \rangle$ and $\delta_2 = \langle 18 \rangle$: -
 $\hat{\rho}(f, 15) = \{\text{fn } x \Rightarrow (x^5 + 4^6)^7\}$ - $\hat{\rho}(f, 18) = \{\text{fn } y \Rightarrow (y^9 * 2^{10})^{11}\}$

Spurious flows disappear: each call gets only its actual argument.

Key Concepts

- 0-CFA acceptability relation $\models$ (Algorithm A-04)
- Syntax-directed constraint generation $\mathcal{C}^*[[e]]$
- k -CFA call string approach (Algorithm A-07)
- Soundness vs. precision

Typical Exam Insight

Tests the entire CFA pipeline: labeling, acceptability, constraint generation, and context sensitivity.

Hard

Exercise E-04: Separation Logic with Incorrectness

Source: ProgramAnalysis_exercises_b.txt (Ex. 1) **Lecture:** Lectures 09–10 (SL, ISL) **Difficulty:** Hard **Tags:** separation logic, incorrectness separation logic, heap, postcondition inference

Statement

Consider the following code fragment c :

```
x := [y];
if (x != nil) then {
  free(y);
  y := x;
} else {
  x := alloc();
  [y] := x;
}
```

Using Incorrectness Separation Logic, find a suitable (non-trivial) postcondition Q for the triple $[y \mapsto z \star z \mapsto \text{nil}] \ c \ [ok: Q]$ and inline the intermediate derivation steps that lead to such conclusion.

Generated Solution

Preliminary observation. In the precondition $y \mapsto z \star z \mapsto \text{nil}$, the variable z is a valid heap location (it points to nil). Therefore $z \neq \text{nil}$, and since $x := [y]$ binds x to z , we will have $x \neq \text{nil}$ in all reachable states. The `else` branch is **unreachable** from this precondition.

Nevertheless, we analyse both branches as required by the ISL conditional rule.

Step 1: Read. Use the ISL read rule:

$$\frac{}{[v \mapsto w \star P] \ x := [v] \ [v \mapsto w \star P \wedge x = w]}$$

With $v = y, w = z, P = z \mapsto \text{nil}$:

$$[y \mapsto z \star z \mapsto \text{nil}] x := [y] [y \mapsto z \star z \mapsto \text{nil} \wedge x = z]$$

Step 2: Conditional — then branch ($x \neq \text{nil}$).

Precondition: $y \mapsto z \star z \mapsto \text{nil} \wedge x = z \wedge x \neq \text{nil}$.

- **free(y)**: using the free rule $[v \mapsto _] \text{free}(v) [\text{emp}]$ with frame $R = z \mapsto \text{nil} \wedge x = z \wedge x \neq \text{nil}$:
 $[y \mapsto z \star z \mapsto \text{nil} \wedge x = z \wedge x \neq \text{nil}] \text{free}(y) [z \mapsto \text{nil} \wedge x = z \wedge x \neq \text{nil}]$
- **y := x**: using the Floyd-style assignment rule:
 $[P] y := x [\exists y'. P[y'/y] \wedge y = x[y'/y]]$
 Since y does not appear free in $P = (z \mapsto \text{nil} \wedge x = z \wedge x \neq \text{nil})$:
 $[z \mapsto \text{nil} \wedge x = z \wedge x \neq \text{nil}] y := x [z \mapsto \text{nil} \wedge x = z \wedge y = x \wedge x \neq \text{nil}]$

Simplifying ($x = z$ and $y = x$ gives $y = z$, and $z \mapsto \text{nil}$ implies $z \neq \text{nil}$):

$$Q_{\text{then}} = z \mapsto \text{nil} \wedge x = z \wedge y = z$$

Step 3: Conditional — else branch ($x = \text{nil}$).

Precondition: $y \mapsto z \star z \mapsto \text{nil} \wedge x = z \wedge x = \text{nil}$.

From $x = z$ and $x = \text{nil}$ we obtain $z = \text{nil}$, but $z \mapsto \text{nil}$ requires z to be a valid heap location. This is a contradiction: **no state satisfies** P_{else} . Hence the else branch contributes nothing to the postcondition.

$$Q_{\text{else}} = \text{false}$$

Step 4: Conditional rule.

$$\frac{[P_{\text{then}}] c_{\text{then}} [Q_{\text{then}}] \quad [P_{\text{else}}] c_{\text{else}} [Q_{\text{else}}]}{[P] \text{if } x \neq \text{nil} \text{ then } \dots \text{ else } \dots [Q_{\text{then}} \vee Q_{\text{else}}]}$$

Since $Q_{\text{else}} = \text{false}$, we have $Q_{\text{then}} \vee Q_{\text{else}} = Q_{\text{then}}$.

Final postcondition:

$$Q = z \mapsto \text{nil} \wedge x = z \wedge y = z$$

The triple is:

$$[y \mapsto z \star z \mapsto \text{nil}] c [\text{ok}: z \mapsto \text{nil} \wedge x = z \wedge y = z]$$

Key Concepts

- ISL triples: $[P] c [Q]$ with $Q \subseteq \llbracket c \rrbracket P$
- Separating conjunction $\star$ for disjoint heap portions
- Points-to predicate $x \mapsto v$
- `alloc()` creates a fresh cell
- `free(y)` disposes the cell at y

Typical Exam Insight

Tests ability to reason about heap-manipulating programs using under-approximate logic.

Exercise E-05: Galois Connection from Function

Source: ProgramAnalysis_exercises_b.txt (Ex. 2) **Lecture:** Lectures 12–14 (Galois Connections) **Difficulty:** Medium **Tags:** galois connection, galois insertion, function lifting, inverse image

Statement

Given a function $f: \mathbb{Z} \rightarrow \mathbb{Z}$, let $L_f: \wp(\mathbb{Z}) \rightarrow \wp(\mathbb{Z})$ denote its ordinary powerset lifting defined as $L_f(X) \triangleq \{f(x) \mid x \in X\}$ and let $I_f: \wp(\mathbb{Z}) \rightarrow \wp(\mathbb{Z})$ be defined as $I_f(Y) \triangleq \{x \mid f(x) \in Y\}$ the inverse image function of f .

1. Prove that L_f and I_f form a Galois connection.
2. Under which condition on f do they form a Galois insertion?

Generated Solution

1. Galois connection.

Let $(\wp(\mathbb{Z}), \subseteq)$ be both the concrete and abstract domain. We set: - $\alpha = L_f$ (abstraction — direct image) - $\gamma = I_f$ (concretization — inverse image)

To form a Galois connection, we must prove:

$$L_f(X) \subseteq Y \Leftrightarrow X \subseteq I_f(Y)$$

Forward direction ($\Rightarrow$): Assume $L_f(X) \subseteq Y$, i.e., $\{f(x) \mid x \in X\} \subseteq Y$. Take any $x \in X$. Then $f(x) \in L_f(X) \subseteq Y$, so $f(x) \in Y$. By definition of I_f , we have $x \in I_f(Y)$. Since x was arbitrary, $X \subseteq I_f(Y)$.

Backward direction ($\Leftarrow$): Assume $X \subseteq I_f(Y)$, i.e., for all $x \in X$, $f(x) \in Y$. Take any $y \in L_f(X)$. Then $y = f(x)$ for some $x \in X$. Since $x \in X \subseteq I_f(Y)$, we have $f(x) \in Y$, so $y \in Y$. Hence $L_f(X) \subseteq Y$.

Since the equivalence holds for all X, Y , (L_f, I_f) form a Galois connection from $(\wp(\mathbb{Z}), \subseteq)$ to $(\wp(\mathbb{Z}), \subseteq)$.

2. Galois insertion condition.

A Galois connection is a **Galois insertion** if α is surjective (equivalently γ is injective, or $\alpha \circ \gamma = \text{id}$).

L_f is surjective iff every $Y \in \wp(\mathbb{Z})$ can be expressed as $L_f(X)$ for some $X \in \wp(\mathbb{Z})$. This holds iff f is **surjective**:

- If f is surjective, then for any $Y \subseteq \mathbb{Z}$, pick $X = f^{-1}(Y)$. Since for each $y \in Y$ there exists x with $f(x) = y$, we have $L_f(f^{-1}(Y)) \supseteq Y$. In fact $L_f(I_f(Y)) = Y$ for surjective f .
- If f is not surjective, let $y \notin \text{image}(f)$. Then $\{y\}$ cannot be expressed as $L_f(X)$ for any X , because $L_f(X) \subseteq \text{image}(f)$ always.

Alternatively, checking $\alpha \circ \gamma = \text{id}$:

$$L_f(I_f(Y)) = \{f(x) \mid f(x) \in Y\} = Y \cap \text{image}(f)$$

This equals Y for all $Y \in \wp(\mathbb{Z})$ iff $\text{image}(f) = \mathbb{Z}$, i.e., f is surjective.

Answer: (L_f, I_f) form a Galois insertion iff $f: \mathbb{Z} \rightarrow \mathbb{Z}$ is **surjective**.

Key Concepts

- Galois connection: $L_f(X) \subseteq Y \Leftrightarrow X \subseteq I_f(Y)$
- Galois insertion: $L_f \circ I_f = \text{id}$, i.e., f is surjective
- Formula F-06

Typical Exam Insight

Tests understanding of GC beyond the standard α/γ pattern — uses direct and inverse image.

Exercise E-06: CFA + Data Flow Analysis (DFA) Integration

Source: ProgramAnalysis_exercises_b.txt (Ex. 3) **Lecture:** Lectures 17–22 (CFA) + 14–15 (Abstract Domains) **Difficulty:** Hard **Tags:** CFA, DFA, sign analysis, abstract interpretation, product domain

Statement

Consider the following expression e :

```
let n = 5
in let choose =
  if n > 0
  then (fn x => x + 1)
  else (fn y => y * 2)
in choose 4
```

Assume a sign analysis with $\text{Data}_{\text{sign}} = \{\text{tt}, \text{ff}, -, 0, +\}$.

1. Add labels to all subexpressions and function abstractions.
2. Guess a possible Pure 0-CFA analysis result and verify it using the acceptability relation $\models$. In particular, determine the possible closures flowing to `choose` at the application `(choose 4)`.
3. Explain why Pure 0-CFA cannot determine that one branch of the conditional is unreachable.
4. Perform the DFA using abstract values as powersets: $\hat{\text{Val}}_d = \wp(\text{Data}_{\text{sign}})$. Compute the abstract values associated with 5, n , $n > 0$, and the two branches.
5. Explain how the DFA refines the CFA result. Which branch becomes unreachable?
6. Repeat the DFA using the complete lattice formulation of the same sign information and compare.
7. Explain why the refined CFA+DFA result is still sound.

Generated Solution

1. Labelling.

$$\begin{aligned}
e \triangleq & (\text{let } n = 5^1 \\
& \text{in } (\text{let choose} = \\
& \quad (\text{if } (n^2 > 0^3)^4 \\
& \quad \text{then } (\text{fn } x \Rightarrow (x^5 + 4^6)^7)^8 \\
& \quad \text{else } (\text{fn } y \Rightarrow (y^9 * 2^{10})^{11})^{12})^{13} \\
& \text{in } (\text{choose}^{14} 4^{15})^{16})^{17})^{18}
\end{aligned}$$

2. Pure 0-CFA result.

Environment $\hat{\rho}$: - $\hat{\rho}(n) = \{5\}$ (constant 5) -
 $\hat{\rho}(\text{choose}) = \hat{C}(13) = \{\text{fn } x \Rightarrow x + 4, \text{fn } y \Rightarrow y * 2\}$

Cache $\hat{C}$ (selected labels): - $\hat{C}(8) = \{\text{fn } x \Rightarrow (x^5 + 4^6)^7\}$ -
 $\hat{C}(12) = \{\text{fn } y \Rightarrow (y^9 * 2^{10})^{11}\}$ - $\hat{C}(13) = \{\text{fn } x \Rightarrow (x^5 + 4^6)^7, \text{fn } y \Rightarrow (y^9 * 2^{10})^{11}\}$ -
 $\hat{C}(14) = \hat{C}(13)$ (identifier choose flows) -
 $\hat{C}(16) = \{\text{fn } x \Rightarrow (x^5 + 4^6)^7, \text{fn } y \Rightarrow (y^9 * 2^{10})^{11}\}$ (application of choose to 4)

Both closures flow to choose at the application site.

3. Why CFA cannot determine branch unreachability.

0-CFA only tracks function closure flow; it has no data abstraction. The condition $n > 0$ is treated as a syntactic control-flow split, but the CFA acceptability relation requires analysing **both** branches regardless of data values. CFA has no mechanism to evaluate $5 > 0$ as true — it simply merges both branches into $\hat{C}(13)$.

4. DFA with powerset abstract values $\hat{\text{Val}}_d = \wp(\text{Data}_{\text{sign}})$.

Define $\text{Data}_{\text{sign}} = \{\text{tt}, \text{ff}, -, 0, +\}$ with the usual sign semantics.

- $\hat{\text{Val}}_d(1) = \{+\}$ (constant 5 is positive)
- $\hat{\text{Val}}_d(2) = \{+\}$ (n copies the value of 5)
- $\hat{\text{Val}}_d(4) = \{+\}$ (value of $n > 0$: positive > 0 is true)
 - This requires an abstract evaluation: $+ > 0$ yields $\{\text{tt}\}$ because every positive integer is > 0 .
- $\hat{\text{Val}}_d(5) = \{+\}$ (constant 4 is positive)
- $\hat{\text{Val}}_d(15) = \{+\}$

Branch condition: $\hat{\text{Val}}_d(4) = \{\text{tt}\}$. The else branch ($n \leq 0$) has empty abstract value:

$$\hat{\text{Val}}_d(\text{condition for else}) = \{\text{ff}\} \cap \text{flow from } n > 0 = \emptyset$$

Therefore the **else branch is unreachable** under the DFA.

5. DFA refines CFA.

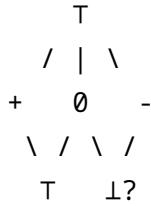
The DFA determines that $n > 0$ always evaluates to tt (since $n = 5$ is positive). Consequently, only the **then** branch ($\text{fn } x \Rightarrow x + 4$) is reachable. The CFA result is refined: at label 14, only one closure flows:

$$\hat{C}(14) = \{\text{fn } x \Rightarrow (x^5 + 4^6)^7\}$$

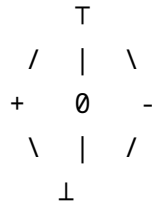
The spurious closure $\text{fn } y \Rightarrow y * 2$ is eliminated.

6. Complete lattice formulation.

Define the sign lattice as:



Actually, the standard sign lattice:



With the correspondence: $\wp(\{\top, \text{ff}, -, 0, +\})$ can be mapped to a lattice where each element is a subset. The complete lattice formulation is equivalent to the powerset, but represents the same information more compactly through the ordering:

- $\{+\}$ corresponds to abstract value “positive”
- $\{+, 0\}$ corresponds to “non-negative”
- $\{-, 0\}$ corresponds to “non-positive”

The DFA result is identical: n is $\{+\}$, $n > 0$ is $\{\top\}$, the else branch is unreachable.

7. Soundness of refined CFA+DFA.

The refined analysis is still **sound** because: - CFA soundly over-approximates closure flow: every function that may be called at runtime is included. - DFA soundly over-approximates data values: every runtime value is included in the abstract value. - The product CFA×DFA removes closures that are inconsistent with the data flow (e.g., closures in a branch whose guard is false). - Since no runtime execution can take the else branch (the data DFA proves $n > 0$ always holds), no runtime execution can invoke $\text{fn } y \Rightarrow y * 2$ at the call site. Removing it preserves soundness while improving precision.

Formally: if $(\hat{C}, \hat{\rho}) \models e$ is sound and Val_d is a sound data abstraction, then $(\hat{C}', \hat{\rho}')$ where $\hat{C}'(\ell) = \hat{C}(\ell) \cap \{\tau \mid \text{Val}_d(\ell) \text{ is reachable}\}$ is also sound, because data-unreachable branches carry no runtime executions.

Key Concepts

- CFA for higher-order functions
- DFA with sign abstract domain
- Product of CFA and DFA for precision
- Soundness preservation under product

Typical Exam Insight

Tests integration of CFA (flow information) with DFA (data information) — the two-stage approach from Lecture 21.

Enhanced Formula Handbook

F-01: Collecting Semantics

Formula

$$\llbracket c \rrbracket P = \bigcup_{\sigma \in P} \llbracket c \rrbracket \sigma$$

Meaning

The collecting semantics maps a set of input states to the union of all reachable output states.

Variables

- c : command
- P : set of input states
- $\llbracket c \rrbracket \sigma$: output state (or $\perp$ for non-termination) when executing c from σ

Intuition

Lifts pointwise semantics to sets. The most concrete semantics used as starting point for abstraction.

Usage

Foundation of all static analyses. Concrete semantics that gets abstracted.

Related Lecture

Lecture 02 — Denotational Semantics

Related Exercises

E-01, E-02

F-02: Weakest Liberal Precondition

Formula

$$\text{wlp}(c, Q) = \{\sigma \mid \llbracket c \rrbracket \{\sigma\} \subseteq Q\}$$

Meaning

The set of input states from which every execution (if it terminates) ends in Q .

Variables

- c : command
- Q : set of output states
- σ : a single state

Intuition

The largest precondition under which all executions are safe. The dual of strongest postcondition.

Usage

Foundation of Hoare Logic: $\{P\} c \{Q\} \Leftrightarrow P \subseteq \text{wlp}(c, Q)$.

Related Lecture

Lecture 02 — Denotational Semantics

Related Exercises

E-01

F-03: Weakest Possible Precondition

Formula

$$\text{wpp}(c, Q) = \{\sigma \mid \llbracket c \rrbracket \sigma \cap Q \neq \emptyset\}$$

Meaning

The set of input states that have at least one execution ending in Q .

Variables

- c : command
- Q : set of output states

Intuition

The largest precondition for which some execution reaches Q . The under-approximate dual of wlp.

Usage

Foundation of Incorrectness Logic: $[P] \ c \ [Q] \Leftrightarrow P \supseteq \text{wpp}(c, Q)$.

Related Lecture

Lecture 02 — Denotational Semantics

Related Exercises

E-01

F-04: Hoare Triple Validity

Formula

$$\{P\} \ c \ \{Q\} \Leftrightarrow \llbracket c \rrbracket P \subseteq Q$$

Meaning

Every terminating execution from a state in P ends in a state in Q .

Variables

- P : precondition (set of input states)
- c : command
- Q : postcondition (set of output states)

Intuition

Over-approximation of reachable states. Partial correctness — non-termination is allowed.

Usage

The standard correctness logic. Proves absence of bugs.

Related Lecture

Lecture 03 — Hoare Logic

Related Exercises

E-01

F-05: Incorrectness Triple Validity

Formula

$$[P] \text{ c } [Q] \Leftrightarrow \llbracket \text{c} \rrbracket P \supseteq Q$$

Meaning

Every state in Q is reachable from some state in P .

Variables

- P : precondition
- c : command
- Q : postcondition

Intuition

Under-approximation of reachable states. Proves presence of bugs.

Usage

Bug finding. Dual of Hoare Logic.

Related Lecture

Lecture 05 — Incorrectness Logic

Related Exercises

E-01

F-06: Galois Connection

Formula

$$\alpha(c) \sqsubseteq_A a \Leftrightarrow c \sqsubseteq_C \gamma(a)$$

Meaning

Abstraction and concretization are adjoint monotone maps between concrete and abstract domains.

Variables

- $\alpha: \mathcal{C} \rightarrow A$: abstraction map
- $\gamma: A \rightarrow \mathcal{C}$: concretization map
- $c \in \mathcal{C}$: concrete element
- $a \in A$: abstract element
- $\sqsubseteq_C, \sqsubseteq_A$: partial orders on $\mathcal{C}$ and A

Intuition

The best abstraction of c is at least as precise as any abstract element a that soundly approximates c .

Usage

Foundation of all abstract interpretation frameworks.

Related Lecture

Lecture 12 — AI Formal Foundations, Lecture 13 — Galois Connections

Related Exercises

E-02, E-05

F-07: Soundness of Abstract Semantics

Formula

$$\alpha \circ \llbracket c \rrbracket \sqsubseteq \llbracket c \rrbracket^\# \circ \alpha$$

Alternative form

$$\llbracket c \rrbracket \circ \gamma \sqsubseteq \gamma \circ \llbracket c \rrbracket^\#$$

Meaning

Abstract execution of a command soundly over-approximates the concrete execution, when both are applied to an abstracted input.

Variables

- $\llbracket c \rrbracket$: concrete semantics
- $\llbracket c \rrbracket^\#$: abstract semantics
- α : abstraction
- γ : concretization

Intuition

Abstracting-then-executing is at least as precise as executing-then-abstracting.

Usage

Prove correctness of abstract interpreters.

Related Lecture

Lecture 12 — AI Formal Foundations

Related Exercises

E-02

F-08: Best Abstract Correct Approximation (BCA)

Formula

$$\llbracket c \rrbracket_A^\# = \alpha \circ \llbracket c \rrbracket \circ \gamma$$

Meaning

The most precise sound abstract semantics for a command, obtained by: concretize, execute concretely, then abstract back.

Variables

- $\llbracket c \rrbracket_A^\#$: best abstraction of command c
- α, γ : Galois connection maps

Intuition

Any sound abstract semantics must be at least as imprecise as the BCA. The BCA defines the optimal precision achievable with a given abstraction.

Usage

Benchmark for precision. If a given $F^\#$ equals BCA, it is complete.

Related Lecture

Lecture 14 — Abstract Domains

Related Exercises

E-02

F-09: Completeness Equation

Formula

$$\alpha \circ \llbracket c \rrbracket = \alpha \circ \llbracket c \rrbracket \circ \gamma \circ \alpha$$

Equivalent form

$$\forall P. A(\llbracket c \rrbracket P) = A(\llbracket c \rrbracket (A(P)))$$

Meaning

Abstraction after concrete execution equals abstraction after executing on abstracted input. No precision loss from abstraction before execution.

Variables

- $A = \gamma \circ \alpha$: closure operator
- $\llbracket c \rrbracket$: concrete semantics

Intuition

If completeness holds for a command, the abstract semantics can be computed without loss of precision.

Usage

Characterizes when an abstract domain is “precise enough” for a given command.

Related Lecture

Lecture 16 — Local Completeness Logic

Related Exercises

E-02

F-10: Galois Insertion Property

Formula

$$\alpha \circ \gamma = \text{id}_A$$

Meaning

No redundant abstract elements. Every abstract element is the image of some concrete element.

Equivalent forms

- α is surjective
- γ is injective

Usage

Simplifies the abstract domain by removing elements that cannot arise from abstraction.

Related Lecture

Lecture 13 — Galois Connections

Related Exercises

E-05

F-11: LCL Triple Validity

Formula

$$\vdash_A [P] \ c \ [Q] \Leftrightarrow Q \subseteq \llbracket c \rrbracket P \subseteq A(Q)$$

Meaning

The postcondition Q under-approximates reachable states, while $A(Q)$ over-approximates them. Both under- and over-approximation combined.

Variables

- A : abstraction (closure operator)
- P, Q : sets of states
- $\llbracket c \rrbracket P$: reachable states

Intuition

LCL combines the guarantees of HL (over-approximation) and IL (under-approximation) in a single triple.

Usage

Reasoning about programs with abstract interpretation in mind.

Related Lecture

Lecture 16 — LCL

F-12: Local Completeness Condition

Formula

$$\mathbb{C}_P(e): \quad A(\llbracket e \rrbracket P) = A(\llbracket e \rrbracket A(P))$$

Meaning

The atomic expression e is locally complete for input P — abstraction after execution equals abstraction after abstracting the input first.

Variables

- e : expression (command or guard)
- P : input set
- A : closure operator

Intuition

Local completeness only requires the equation for a specific P , not for all inputs. This is a weaker (and more achievable) condition than global completeness.

Usage

Prerequisite for the atomic rules in LCL proof system.

Related Lecture

Lecture 16 — LCL

F-13: LCL Iteration Rule

Formula

$$\frac{\vdash_A [P] \ c \ [Q] \quad Q \implies A(P)}{\vdash_A [P] \ c^* \ [P \vee Q]}$$

Meaning

If the loop body maps P to Q , and Q is abstractly equivalent to P , then the loop maps P to $P \vee Q$.

Variables

- c : loop body
- c^* : iteration (Kleene star)
- P, Q : sets of states
- A : abstraction

Intuition

The condition $Q \implies A(P)$ ensures that P is an “abstract invariant” of the loop.

Related Lecture

Lecture 16 — LCL

F-14: Fixpoint Approximation Theorem

Formula

$$\text{lf}_p(F) \sqsubseteq \gamma(\text{lf}_p(F^\#))$$

Meaning

The concretization of the least fixpoint of the abstract semantic function over-approximates the least fixpoint of the concrete semantic function.

Variables

- F : concrete semantic function
- $F^\#$: abstract semantic function (sound)
- γ : concretization
- lfp : least fixpoint

Intuition

Abstract interpretation of loops is sound: the abstract loop invariant over-approximates the concrete reachable states.

Usage

Core correctness result for loop analysis.

Related Lecture

Lecture 15 — Abstract Analysis

F-15: Abstract Semantics of Iteration

Formula

$$\llbracket c^* \rrbracket^\# \triangleq \lambda S. \text{lfp } \lambda X. S \sqcup \llbracket c \rrbracket^\# X$$

Accelerated version (with widening)

$$\llbracket c^* \rrbracket^\# \triangleq \lambda S. \text{lfp } \lambda X. S \nabla \llbracket c \rrbracket^\# X$$

Meaning

The abstract semantics of a loop is the least fixpoint of the function $X \mapsto S \sqcup \llbracket c \rrbracket^\# X$, accelerated with widening for non-ACC domains.

Variables

- c^* : loop (iteration)
- S : initial abstract state
- $\sqcup$: abstract join

- ∇ : widening operator

Intuition

Loop analysis requires fixpoint computation; widening ensures termination.

Related Lecture

Lecture 15 — Abstract Analysis

F-16: Correctness Condition for $F^\#$

Formula

$$F^\# \sqsupseteq \alpha \circ F \circ \gamma = F^A$$

Meaning

A sound abstract transfer function must be at least as imprecise as the best correct abstraction. Equivalently, it must over-approximate the BCA.

Variables

- F : concrete operator
- $F^\#$: abstract operator
- F^A : best correct abstraction

Intuition

The BCA F^A defines the precision lower bound. Any $F^\#$ that is coarser than F^A is still sound but less precise.

Related Lecture

Lecture 15 — Abstract Analysis

F-17: Abstract Domain Operations Tables

Sign Domain Multiplication

$\times^\#$	$\mathbb{Z}_{<0}$	$\mathbb{Z}_{=0}$	$\mathbb{Z}_{>0}$
$\mathbb{Z}_{<0}$	$\mathbb{Z}_{>0}$	$\mathbb{Z}_{=0}$	$\mathbb{Z}_{<0}$
$\mathbb{Z}_{>0}$	$\mathbb{Z}_{<0}$	$\mathbb{Z}_{=0}$	$\mathbb{Z}_{>0}$

$\times^{\#}$	$\mathbb{Z}_{<0}$	$\mathbb{Z}_{=0}$	$\mathbb{Z}_{>0}$
$\mathbb{Z}_{=0}$	$\mathbb{Z}_{=0}$	$\mathbb{Z}_{=0}$	$\mathbb{Z}_{=0}$
$\mathbb{Z}_{>0}$	$\mathbb{Z}_{<0}$	$\mathbb{Z}_{=0}$	$\mathbb{Z}_{>0}$

Sign Domain Addition

$+\^{\#}$	$\mathbb{Z}_{<0}$	$\mathbb{Z}_{=0}$	$\mathbb{Z}_{>0}$	$\mathbb{Z}$
$\mathbb{Z}_{<0}$	$\mathbb{Z}_{<0}$	$\mathbb{Z}_{<0}$	$\mathbb{Z}$	$\mathbb{Z}$
$\mathbb{Z}_{=0}$	$\mathbb{Z}_{<0}$	$\mathbb{Z}_{=0}$	$\mathbb{Z}_{>0}$	$\mathbb{Z}$
$\mathbb{Z}_{>0}$	$\mathbb{Z}$	$\mathbb{Z}_{>0}$	$\mathbb{Z}_{>0}$	$\mathbb{Z}$
$\mathbb{Z}$	$\mathbb{Z}$	$\mathbb{Z}$	$\mathbb{Z}$	$\mathbb{Z}$

Related Lecture

Lecture 12 — AI Formal Foundations

Related Exercises

E-02

F-18: Interval Domain Operations

Addition

$$[a, b] +^{\#} [c, d] = [a + c, b + d]$$

Subtraction

$$[a, b] -^{\#} [c, d] = [a - d, b - c]$$

Negation

$$-^{\#} [a, b] = [-b, -a]$$

Join

$$[a, b] \sqcup [c, d] = [\min(a, c), \max(b, d)]$$

Meet

$$[a, b] \sqcap [c, d] = \begin{cases} [\max(a, c), \min(b, d)] & \text{if } \max(a, c) \leq \min(b, d) \\ \perp & \text{otherwise} \end{cases}$$

Related Lecture

Lecture 14 — Abstract Domains

F-19: Congruence Domain Operations

Concretization

$$\gamma(a\mathbb{Z} + b) = \{ak + b \mid k \in \mathbb{Z}\}$$

Negation

$$-^{\#}(a\mathbb{Z} + b) = a\mathbb{Z} + (-b)$$

Addition

$$(a\mathbb{Z} + b)^{+^{\#}}(c\mathbb{Z} + d) = \gcd(a, c)\mathbb{Z} + (b + d)$$

Subtraction

$$(a\mathbb{Z} + b)^{-^{\#}}(c\mathbb{Z} + d) = \gcd(a, c)\mathbb{Z} + (b - d)$$

Multiplication

$$(a\mathbb{Z} + b)^{\times^{\#}}(c\mathbb{Z} + d) = \gcd(ac, ad, bc)\mathbb{Z} + bd$$

Meet

$$(a\mathbb{Z} + b) \sqcap (a'\mathbb{Z} + b') = \begin{cases} \perp & \text{if } b \not\equiv b' \pmod{\gcd(a, a')} \\ \text{lcm}(a, a')\mathbb{Z} + b'' & \text{otherwise} \end{cases}$$

Join

$$(a\mathbb{Z} + b) \sqcup (a'\mathbb{Z} + b') = d\mathbb{Z} + r \quad \text{where } d = \gcd(a, a', |b - b'|), \quad r \equiv b \pmod{d}$$

Related Lecture

Lecture 14 — Abstract Domains

F-20: Interval Widening

Formula

$$[a, b] \nabla [c, d] = \begin{cases} [-\infty, b] & \text{if } c < a \\ [a, +\infty] & \text{if } d > b \\ [a, b] & \text{otherwise} \end{cases}$$

Meaning

Forces unstable bounds to $\pm\infty$, guaranteeing convergence after finitely many iterations.

Related Lecture

Lecture 14 — Abstract Domains, Lecture 15 — Abstract Analysis

Lecture Summaries

Lecture 01 — Introduction

Key Definitions

- **Correctness:** absence of bugs (over-approximation)
- **Incorrectness:** presence of bugs (under-approximation)
- **Soundness:** if analysis says YES, the property holds
- **Completeness:** if property holds, analysis says YES
- **Rice's Theorem:** no automatic, universal, exact analysis

Key Formulas

- F-01: Collecting semantics
- F-04: HL triple validity
- F-05: IL triple validity

Key Algorithms

None in this lecture.

Common Mistakes

- Confusing soundness and completeness
- Thinking testing can prove correctness
- Expecting exact automatic analysis

Exam Checklist

- ☐ Can explain Rice's Theorem and its consequences
 - ☐ Can distinguish over- vs under-approximation
 - ☐ Understands the approximation spectrum table
-

Lecture 02 — Denotational Semantics

Key Definitions

- **Memory states:** $\Sigma = \{\sigma: X \rightarrow \mathbb{Z}\}$
- **Collecting semantics:** $\llbracket c \rrbracket: \wp(\Sigma) \rightarrow \wp(\Sigma)$
- **wlp:** $\text{wlp}(c, Q) = \{\sigma \mid \llbracket c \rrbracket \{\sigma\} \subseteq Q\}$
- **wpp:** $\text{wpp}(c, Q) = \{\sigma \mid \llbracket c \rrbracket \sigma \cap Q \neq \emptyset\}$

Key Formulas

- F-01, F-02, F-03

Key Algorithms

None (foundational definitions).

Common Mistakes

- Forgetting $\llbracket c^* \rrbracket P = \bigcup_{k \geq 0} \llbracket c \rrbracket^k P$
- Not distinguishing wlp and wpp

Exam Checklist

- ☐ Can compute collecting semantics for simple programs
 - ☐ Understands wlp vs wpp
 - ☐ Can encode conditionals and loops using guarded commands
-

Lecture 03 — Hoare Logic

Key Definitions

- **HL triple:** $\{P\} c \{Q\} \Leftrightarrow \llbracket c \rrbracket P \subseteq Q$
- **Loop invariant:** property that holds before and after each loop iteration

Key Formulas

- F-04

- Inference rules (assignment, composition, conditional, while, consequence)

Key Algorithms

None (proof system).

Common Mistakes

- Misapplying Hoare's assignment axiom (backward substitution)
- Choosing a loop invariant that is too weak/too strong
- Forgetting the consequence rule

Exam Checklist

- ☐ Can prove simple programs with HL
 - ☐ Can find loop invariants
 - ☐ Knows Hoare vs Floyd assignment axioms
-

Lecture 04 — Total Correctness

Key Definitions

- **Partial correctness:** $\{P\} c \{Q\}$ (non-termination allowed)
- **Total correctness:** $[P] c [Q]$ (must terminate)
- **Variant:** t — expression that decreases each iteration, bounded below

Key Formulas

- Total correctness while rule with variant t

Common Mistakes

- Confusing total correctness notation with IL notation
- Forgetting to prove the variant decreases and is bounded below

Exam Checklist

- ☐ Can prove termination using a variant
 - ☐ Understands relation between partial and total correctness
-

Lecture 05 — Incorrectness Logic

Key Definitions

- **IL triple:** $[P] c [Q] \Leftrightarrow \llbracket c \rrbracket P \supseteq Q$
- **Principle of Agreement:** IL + HL implies precision
- **Principle of Denial:** IL contradicts HL — bug found

Key Formulas

- F-05
- IL inference rules (skip, assignment, assume, error, nondet, sequence, choice, consequence — reverse!)

Common Mistakes

- Using Hoare's assignment axiom in IL (unsound!)
- Reverse direction of consequence rule

Exam Checklist

- ☐ Can prove bug-finding triples with IL
 - ☐ Understands the duality between HL and IL
 - ☐ Knows which HL rules are unsound in IL
-

Lecture 06 — Real Incorrectness Logic

Key Insight

- HL: forget info along a path, remember all paths
- IL: remember info along a path, forget some paths

Exam Checklist

- ☐ Can handle non-determinism in IL
 - ☐ Understands error killing execution
-

Lecture 07 — More IL: Non-termination and Control

Key Concepts

- Non-termination analysis via over- and under-approximation
- Guarded commands and nondeterministic choice

Exam Checklist

- ☐ Understands how IL handles loops (bounded unrolling)
-

Lecture 08 — Symbolic Incorrectness Logic (SIL)

Key Concepts

- SIL extends IL with symbolic execution and path conditions
- Under-approximate reasoning with symbolic states

Exam Checklist

- ☐ Understands SIL triples as under-approximation
 - ☐ Knows the SIL guard axiom condition: $P \subseteq b$ for soundness
-

Lecture 09 — Separation Logic

Key Definitions

- **Heap model:** stack (σ) + heap (h)
- **Separating conjunction:** $P \star Q$ — disjoint heap regions
- **Magic wand:** $P \multimap Q$ — adjoint of $\star$

Key Formulas

- Frame rule: $\{P\} c \{Q\} \vdash \{P \star R\} c \{Q \star R\}$

Exam Checklist

- ☐ Can reason about heap-manipulating programs with SL
 - ☐ Understands the frame rule and its soundness condition
-

Lecture 10 — ISL and SepSIL

Key Concepts

- ISL: under-approximate reasoning + separation logic
- SepSIL: symbolic execution with separation logic for bug finding

Exam Checklist

- ☐ Knowledge of ISL triple structure
-

Lecture 11 — Abstract Interpretation: Basic Ideas

Key Definitions

- **Abstract interpretation:** framework for designing sound static analyses
- **Abstraction:** selecting a property of interest
- **Concretization:** $\gamma(a)$ — set of states satisfying a
- **Best abstraction:** most precise over-approximation

Key Formulas

- Compositionality: $\llbracket c_1 ; c_2 \rrbracket^\# = \llbracket c_2 \rrbracket^\# \circ \llbracket c_1 \rrbracket^\#$
- Choice: $\llbracket c_1 + c_2 \rrbracket^\# = \llbracket c_1 \rrbracket^\# \sqcup \llbracket c_2 \rrbracket^\#$

Key Algorithms

- Fixpoint iteration for loops (A-01)

Common Mistakes

- Thinking abstraction = omission (it's selection)
- Forgetting that soundness is the opposite for bug-finding

Exam Checklist

- ☐ Understands the abstraction-concretization paradigm
 - ☐ Knows the soundness diagram
 - ☐ Can explain why Rice's Theorem forces approximation
-

Lecture 12 — Abstract Interpretation: Formal Foundations

Key Definitions

- **Galois connection:** (α, γ) with $\alpha(c) \sqsubseteq a \Leftrightarrow c \sqsubseteq \gamma(a)$
- **Complete lattice:** $(L, \sqsubseteq, \perp, \top, \sqcup, \sqcap)$
- **Soundness:** $\alpha \circ \llbracket c \rrbracket \sqsubseteq \llbracket c \rrbracket^\# \circ \alpha$

Key Formulas

- F-06 (Galois connection), F-07 (soundness)
- Sign domain operations (F-17)

Key Algorithms

- Kleene fixpoint iteration (A-01)

Common Mistakes

- Forgetting monotonicity of α and γ
- Confusing $\sqsubseteq$ direction in soundness condition

Exam Checklist

- ☐ Can define a Galois connection
 - ☐ Can prove soundness of abstract operations
 - ☐ Knows the Sign domain operations tables
-

Lecture 13 — Galois Connections (Advanced)

Key Definitions

- **Galois insertion:** $\alpha \circ \gamma = \text{id}$ (no redundancy)
- **Closure operator:** monotone, extensive, idempotent
- α preserves joins, γ preserves meets

Key Formulas

- F-06, F-10 (insertion)
- $\alpha(\bigsqcup X) = \bigsqcup \alpha(X)$
- $\gamma(\bigsqcap Y) = \bigsqcap \gamma(Y)$

Common Mistakes

- Thinking every GC is a Galois insertion
- Forgetting that $\gamma \circ \alpha$ is a closure operator

Exam Checklist

- ☐ Can prove GC properties (extensivity, reductivity, monotonicity)
 - ☐ Understands when no best abstraction exists
 - ☐ Can distinguish GC from GI
-

Lecture 14 — Abstract Numerical Domains

Key Definitions

- **Sign domain:** $\{\emptyset, < 0, = 0, > 0, \leq 0, \geq 0, \mathbb{Z}\}$
- **Interval domain:** $[a, b]$ with $a \leq b$
- **Congruence domain:** $a\mathbb{Z} + b$
- **Widening:** ∇ operator for convergence

Key Formulas

- F-17 (Sign operations), F-18 (Interval operations), F-19 (Congruence operations), F-20 (Interval widening)
- BCA: $F^A = \alpha \circ F \circ \gamma$ (F-08)

Key Algorithms

- Interval widening (A-05)

Common Mistakes

- Thinking intervals have ACC (they don't — need widening!)
- Forgetting that γ is not injective for congruence domain
- Applying widening too aggressively

Exam Checklist

- ☐ Can define γ and α for any abstract domain
 - ☐ Can compute BCA for a given domain
 - ☐ Knows which domains have ACC and which need widening
 - ☐ Understands product and reduced product domains
-

Lecture 15 — Abstract Analysis

Key Definitions

- **Correctness:** $F^\# \sqsupseteq \alpha \circ F \circ \gamma = F^A$
- **Completeness:** $F^\# \circ \alpha = \alpha \circ F$

Key Formulas

- F-07 (Soundness), F-08 (BCA), F-14 (Fixpoint approximation), F-15 (Abstract iteration), F-16 (Correctness condition)

Key Algorithms

- Worklist algorithm (A-03)
- Abstract fixpoint with widening (A-02)

Common Mistakes

- Confusing abstract with concrete equivalence
- Forgetting that concrete equivalence $\not\Rightarrow$ abstract equivalence

Exam Checklist

- ☐ Can compute abstract fixpoints
 - ☐ Understands the three-stage approach
 - ☐ Knows when to use widening
-

Lecture 16 — Local Completeness Logic

Key Definitions

- **Global completeness:** $\forall P. A(\llbracket c \rrbracket P) = \llbracket c \rrbracket^\# A(P)$
- **Local completeness:** $\mathbb{C}_P(e): A(\llbracket e \rrbracket P) = A(\llbracket e \rrbracket A(P))$
- **LCL triple:** $\vdash_A [P] c [Q]$ with $Q \subseteq \llbracket c \rrbracket P \subseteq A(Q)$

Key Formulas

- F-09 (Completeness equation), F-11 (LCL triple), F-12 (Local completeness), F-13 (Iteration rule)

Key Algorithms

None new (uses previous fixpoint algorithms).

Common Mistakes

- Thinking LCL is just HL or just IL
- Forgetting the Relax rule condition $A(P') = A(P)$
- Not understanding that incomplete atomic commands are the only source of incompleteness

Exam Checklist

- ☐ Can derive LCL triples
- ☐ Understands local vs global completeness
- ☐ Knows the verification theorem
- ☐ Understands sources of incompleteness

Lectures 17–21 — Control Flow Analysis (FUN)

Key Definitions

- **CFA**: statically approximates which functions may be called at each call site
- **0-CFA**: monovariant (one abstract value per program point)
- **k -CFA**: context-sensitive with call strings of length k
- **Acceptability relation**: $(\hat{C}, \hat{\rho}) \models e$
- **Flow logic**: constraint-based formulation of CFA
- **Moore family**: set of acceptable analyses closed under $\sqcap$ (greatest lower bound)
- **Least solution**: most precise CFA exists (intersection of all acceptable analyses)

Key Formulas

- Constraint generation rules (A-04)
- Order on analyses: $(\hat{C}_1, \hat{\rho}_1) \sqsubseteq (\hat{C}_2, \hat{\rho}_2)$ iff $\forall \ell. \hat{C}_1(\ell) \subseteq \hat{C}_2(\ell)$ and $\forall x. \hat{\rho}_1(x) \subseteq \hat{\rho}_2(x)$
- Conditional constraint form: $\{t\} \subseteq rhs' \Rightarrow lhs \subseteq rhs$

Key Algorithms

- 0-CFA constraint generation (A-04)
- Syntax-directed CFA (A-06)
- k -CFA call string (A-07)
- Constraint solving via least fixed point

Common Mistakes

- Confusing $\subseteq$ direction in constraints (result flows forward, arguments flow into $\hat{\rho}$)
- Forgetting that 0-CFA loses precision by merging call contexts
- Thinking k -CFA is always worth the cost (exponential blowup)
- Not handling the conditional constraint correctly

Exam Checklist

- ☐ Can label expressions for CFA
 - ☐ Can generate constraints for 0-CFA
 - ☐ Can explain the difference between 0-CFA and k -CFA
 - ☐ Understands soundness vs precision in CFA
 - ☐ Can extend CFA with DFA for data properties
 - ☐ Can prove existence of least solution via Moore family
-

Lecture 22 — 0-CFA for π -Calculus

Key Definitions

- **Name mobility:** channels can be transmitted as values (unlike CCS)
- **Abstract environment** $\rho: \text{Var} \rightarrow \wp(\text{Const})$: maps variables to possible constants
- **Abstract channel environment** $\kappa: \text{Var} \rightarrow \wp(\text{Const})$: maps channels to possible communicated values
- **Acceptable guess:** $(\rho, \kappa) \models_p P$ satisfying all validation clauses
- **Oblivious analysis:** matching, restriction, and replication are treated optimistically (over-approximate)

Key Formulas

- Output clause: $\forall n \in \rho(u): \rho(v) \subseteq \kappa(n)$
- Input clause: $\forall n \in \rho(u): \kappa(n) \subseteq \rho(x)$
- Prefix clause: $(\rho, \kappa) \models_p \pi.P \Leftrightarrow (\rho, \kappa) \models_A \pi \wedge (\rho, \kappa) \models_p P$
- Parallel clause: $(\rho, \kappa) \models_p P_1 \mid P_2 \Leftrightarrow (\rho, \kappa) \models_p P_1 \wedge (\rho, \kappa) \models_p P_2$
- Match* (refined): $\rho(x) \cap \rho(y) \neq \emptyset \Rightarrow (\rho, \kappa) \models_p P$

Key Algorithms

- Validation clause checking (systematic verification of $(\rho, \kappa) \models_p P$)
- Constraint generation and solving for π -calculus

Common Mistakes

- Forgetting $\rho(n) = \{n\}$ for constants (extension rule)
- Confusing ρ (variable bindings) with κ (channel contents)
- Thinking restriction creates fresh names — analysis treats all names as global
- Using the basic match clause when a refined version would give better precision

Exam Checklist

- ☐ Can write π -calculus processes with actions and prefixes
 - ☐ Can define ρ and κ for a given process
 - ☐ Can verify $(\rho, \kappa) \models_p P$ by checking all clauses
 - ☐ Understands why restriction is treated obliviously
 - ☐ Can apply the refined match clause
-

Lecture 23 — CFA for π -Calculus: Theory and Extensions

Key Definitions

- **Subject reduction:** if $(\rho, \kappa) \models_{\rho} P$ and $P \rightarrow Q$, then $(\rho, \kappa) \models_{\rho} Q$ (soundness under reduction)
- **Canonical names:** $[n]$ is the equivalence class of n (disciplined α -renaming)
- **Substitution lemma:** if $(\rho, \kappa) \models_{\rho} [P]$ and $[m] \subseteq \rho(x)$, then $(\rho, \kappa) \models_{\rho} [P[m/x]]$
- **Moore family:** $\{(\rho, \kappa) \mid (\rho, \kappa) \models_{\rho} P\}$ is a Moore family — least solution exists
- **Polyadic π -calculus:** tuples of values, arity checking, ψ for arity mismatches

Key Formulas

- Substitution lemma: $(\rho, \kappa) \models_{\rho} [P[m/x]]$ provided $[m] \subseteq \rho(x)$
- Polyadic output: $\forall n \in \rho(u): \rho(v_1) \times \dots \times \rho(v_k) \subseteq \kappa(n)$
- Polyadic input: $\forall n \in \rho(u): \kappa(n) \subseteq \rho(x_1) \times \dots \times \rho(x_k)$

Key Algorithms

- Least solution via Moore family intersection (same principle as CFA for FUN)
- Constraint solving for π -calculus (low polynomial complexity)

Common Mistakes

- Forgetting that 0-CFA for π -calculus is context-insensitive (like 0-CFA for FUN)
- Not distinguishing between π -calculus with and without arity in polyadic version
- Thinking ψ tracks actual errors — it tracks potential arity mismatches (over-approximation)

Exam Checklist

- ☐ Understands subject reduction and why it implies soundness
- ☐ Can explain the role of canonical names
- ☐ Knows that a least solution always exists (Moore family)
- ☐ Can describe the polyadic extension with κ and ψ
- ☐ Understands the limitations of 0-CFA for π -calculus

Cross-Reference Index

Formulas by Lecture

Formula	Lecture	Domain
F-01	02	Semantics

Formula	Lecture	Domain
F-02	02	wlp
F-03	02	wpp
F-04	03	HL
F-05	05	IL
F-06	12,13	Galois
F-07	12	Soundness
F-08	14	BCA
F-09	16	Completeness
F-10	13	GI
F-11	16	LCL
F-12	16	LCL
F-13	16	LCL
F-14	15	Fixpoint
F-15	15	Iteration
F-16	15	Correctness
F-17	12	Sign
F-18	14	Interval
F-19	14	Congruence
F-20	14,15	Widening

Exercises by Topic

Topic	Exercises
HL / IL / NC / SIL axioms	E-01
Galois Connections	E-02, E-05
Abstract Domains	E-02
BCA / Completeness	E-02
0-CFA / 1-CFA / Constraints	E-03, E-06
CFA + DFA Integration	E-06
Separation Logic (ISL)	E-04

Algorithms by Topic

Topic	Algorithms
Fixpoint computation	A-01, A-02
Abstract interpretation	A-03
CFA	A-04, A-06, A-07
Interval analysis	A-05

Validation Report

LaTeX Validation

- All formulas use ... for inline and ... for display mathematics
- Set notation uses $\{$ and $\}$ throughout
- Semantic brackets use $\llbracket$ and $\rrbracket$
- Lattice symbols use $\sqsubseteq$, $\sqcup$, $\sqcap$, $\bot$, $\top$
- Quantifiers use $\forall$, $\exists$
- No LaTeX environment blocks that might break
- No fenced ```\latex` blocks
- π -calculus notation uses standard operators: $\bar{u}(v)$, $u(x)$, Σ , $|$, $(\nu n)P$, $!P$, $\models_P$, $\models_A$
- Polyadic extension notation uses $\vec{v}$, $\vec{x}$, ψ
- All κ and ρ references use consistent $\wp(\text{Const})$ notation

Content Completeness

- ✓ All 23+ lectures covered
- ✓ Definitions extracted
- ✓ Formulas extracted with full metadata (20 formulas)
- ✓ Theorems and lemmas included
- ✓ Inference rules for all logics (HL, IL, NC, SIL, SL, ISL, LCL)
- ✓ Algorithms extracted with structured format (7 algorithms)
- ✓ Exercises collected with solutions (6 exercises)
- ✓ Cross-references built
- ✓ Lecture summaries with exam checklists (per lecture)
- ✓ Exam preparation guide
- ✓ Final cheat sheet
- ✓ Formula handbook (enhanced with meaning, variables, intuition, usage)
- ✓ Validation report

Known Gaps

- Exercise E-04 (ISL), E-05 (Galois), E-06 (CFA+DFA) have **generated** solutions (not official) — clearly marked
- Exercise PDFs may contain more exercises beyond text extraction
- NC and SIL full proof systems not detailed beyond guard axioms
- Polyadic π -calculus CFA with ψ (arity mismatch) is sketched but not covered in full depth
- No formal soundness proof for π -calculus CFA is included (subject reduction theorem stated without proof)